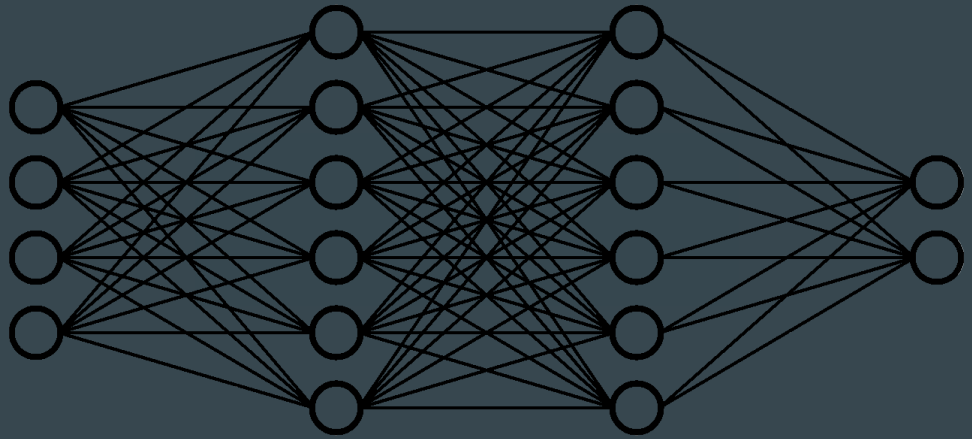
Sistemas de Inteligencia Artificial: TP3 - Perceptrón Simple y Multicapa



ITBA

Franco Morroni (60417)
Bernardo Zapico (62318)

Ejercicio 1 - Perceptrón Simple Escalón

Buscamos comparar las diferencias en el entrenamiento del perceptrón según:

- La tasa de aprendizaje
- Forma de inicialización de pesos (aleatorios o en cero)

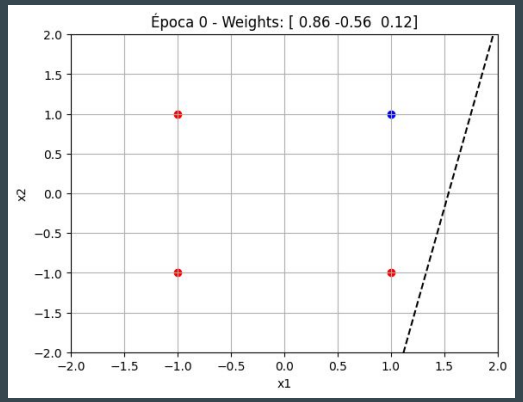
Ejercicio 1 - Función AND

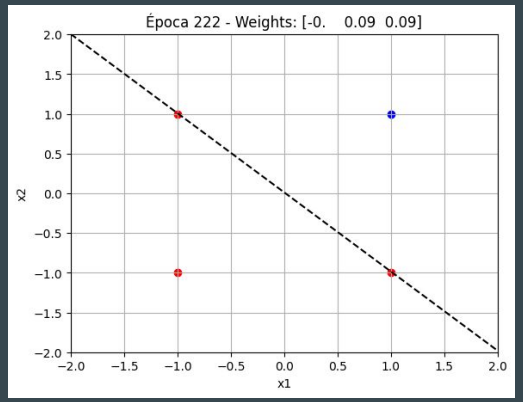
Queremos clasificar las posibles entradas de la función AND según la salida de la función:

X1	X2	Salida
-1	-1	-1
-1	1	-1
1	-1	-1
1	1	1

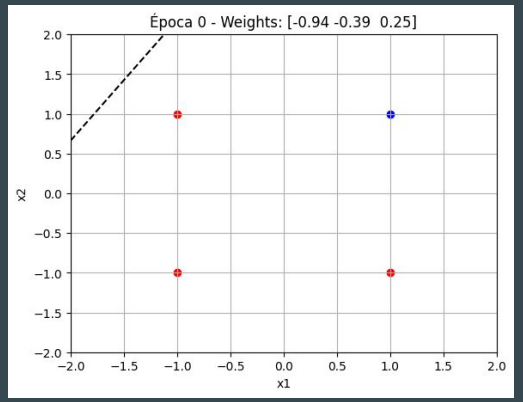
Función AND - Variación en tasa de aprendizaje - Inicialización aleatoria

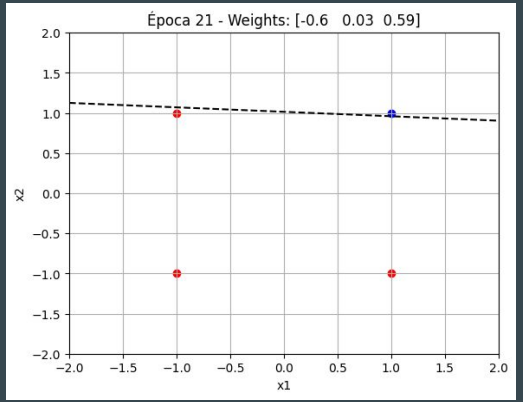
Learning rate: 0.001



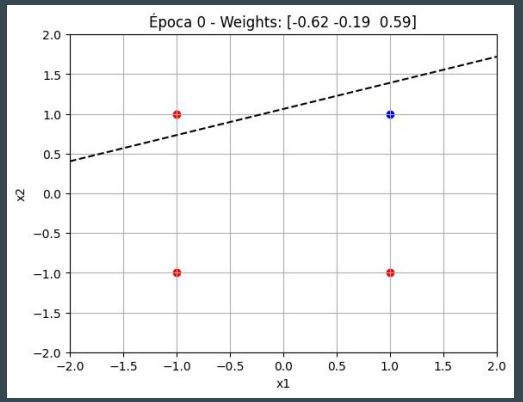


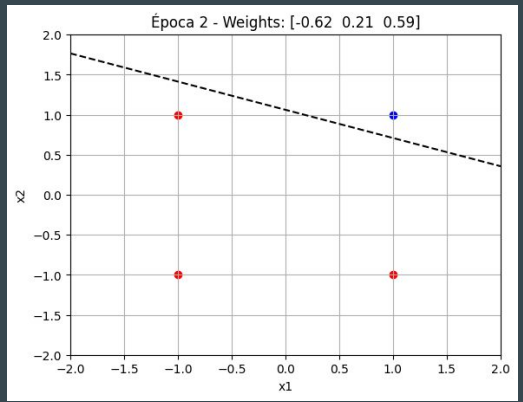
Learning rate: 0.01





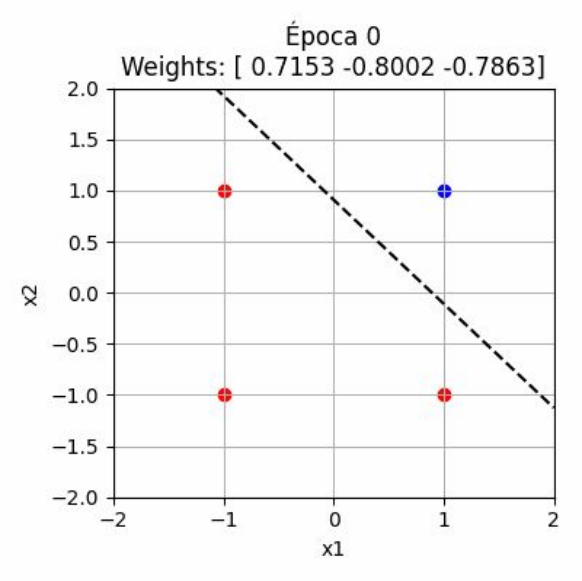
Learning rate: 0.1



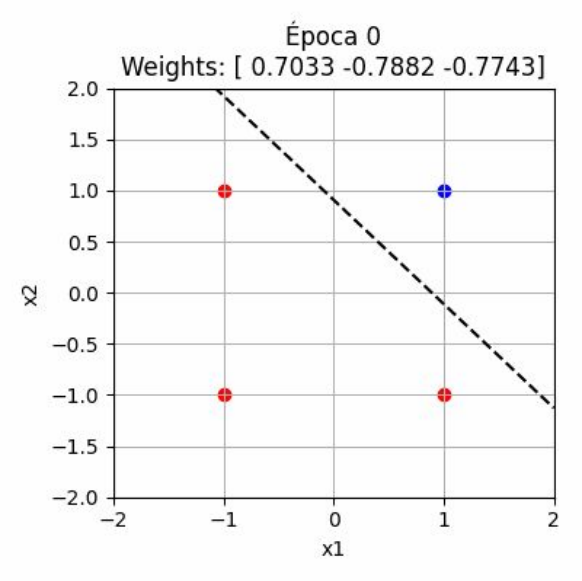


Función AND - Variación en tasa de aprendizaje

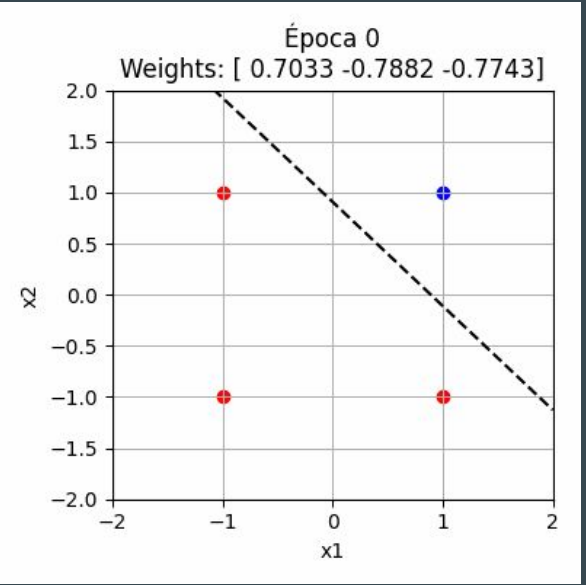
Learning rate: 0.001



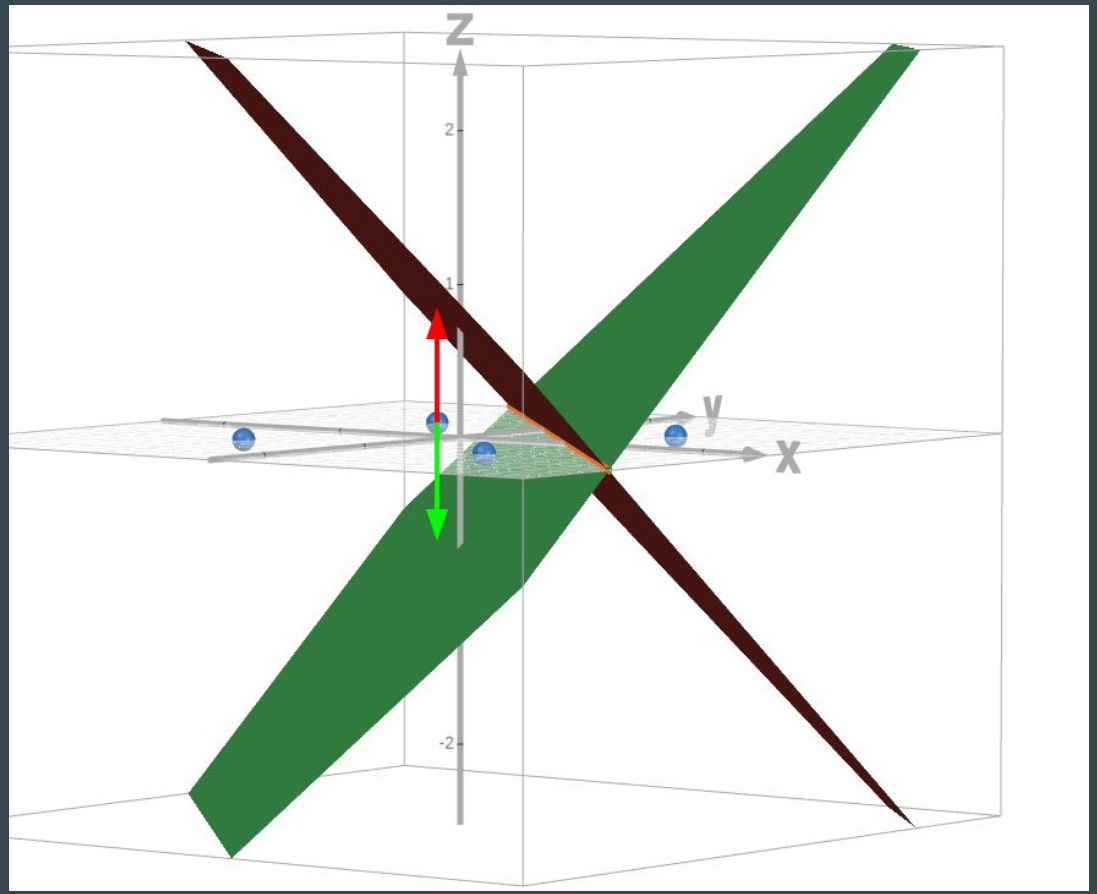
Learning rate: 0.01



Learning rate: 0.1

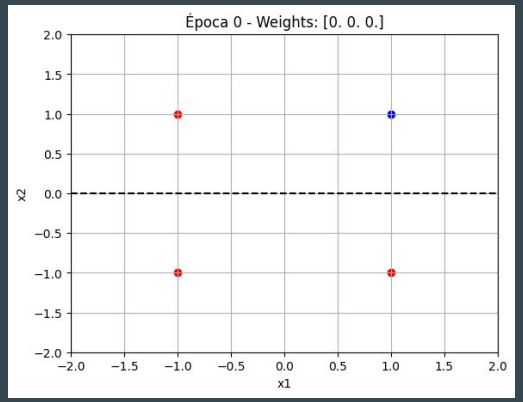


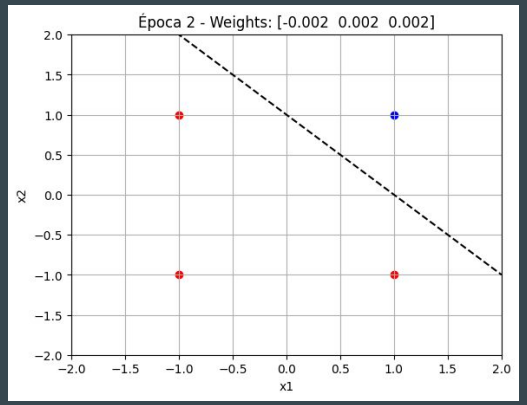
Significado de la salida del perceptrón



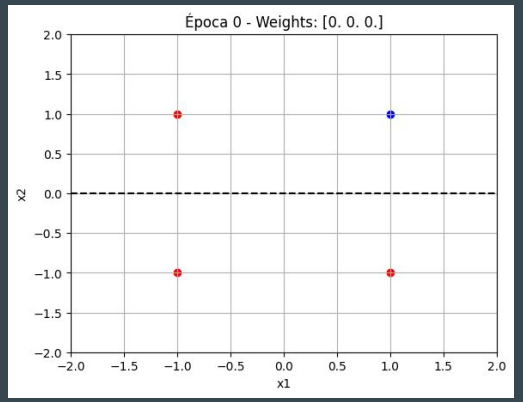
Función AND - Variación en tasa de aprendizaje - Inicialización en cero

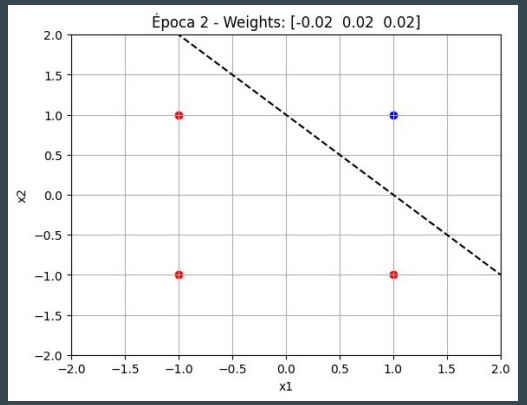
Learning rate: 0.001



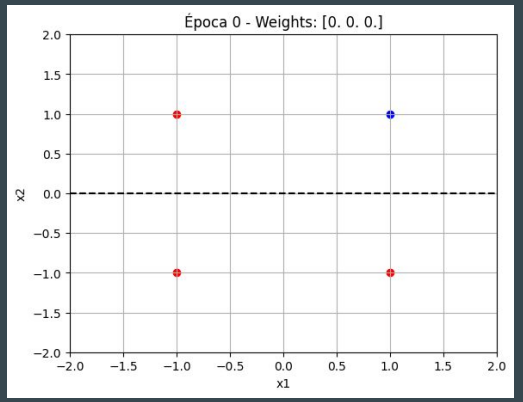


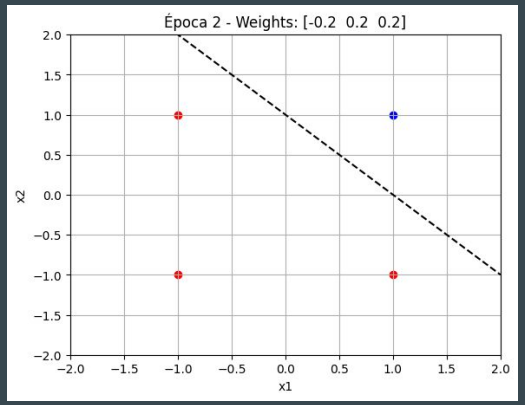
Learning rate: 0.01





Learning rate: 0.1



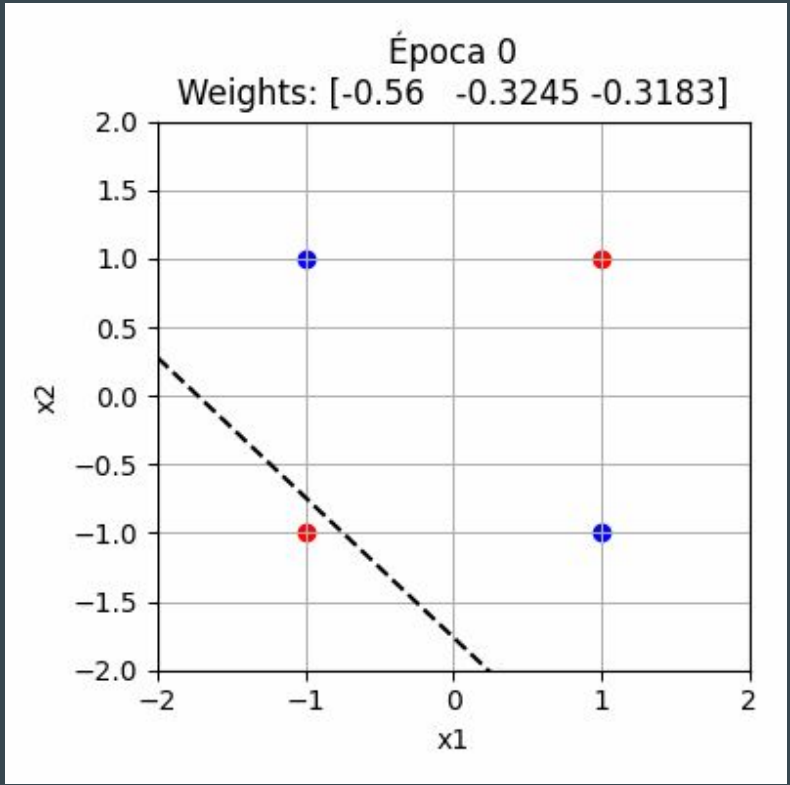


Ejercicio 1 - Función XOR

Queremos clasificar las posibles entradas a la función XOR según la salida de la función:

X1	X2	Salida
-1	-1	-1
-1	1	1
1	-1	1
1	1	-1

Función XOR


Época 0
Weights: [-0.56 -0.3245 -0.3183]
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
x2
−2
−1
0
1
2
x1

Conclusión

El perceptrón simple escalón puede resolver problemas de clasificación binaria pero no es útil si las clases no pueden ser separadas por un hiperplano. Es decir, si no son dos clases linealmente separables.

Ejercicio 2 - Perceptrón Lineal y no Lineal

Para el dado conjunto de datos buscamos comparar las soluciones según:

- La función de activación (identidad, tangente hiperbólica, logística)
- La tasa de aprendizaje
- El número máximo de épocas
- La estrategia de cambio de pesos
- β (para no lineales)
- Forma de inicialización de pesos (aleatorios o en cero)
- Generalización

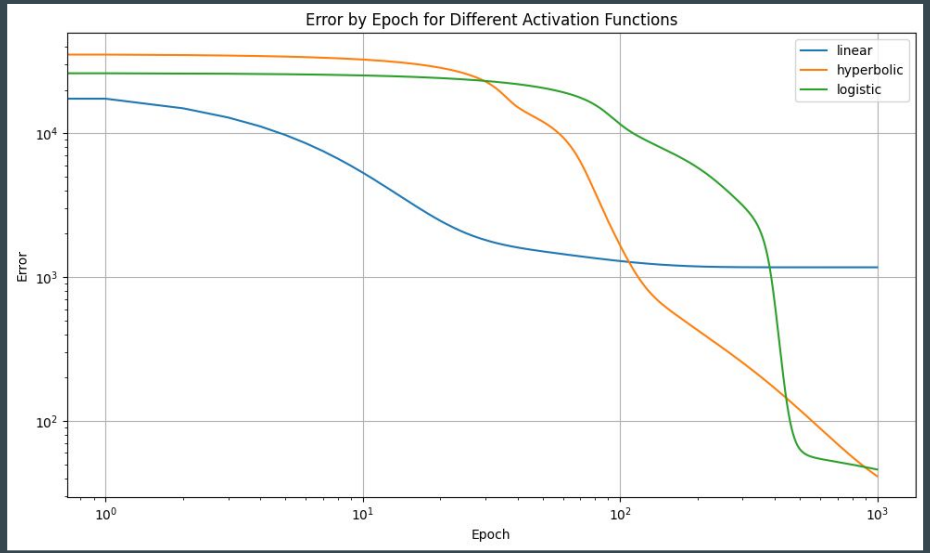
Ejercicio 2 - Perceptrón Lineal y no Lineal

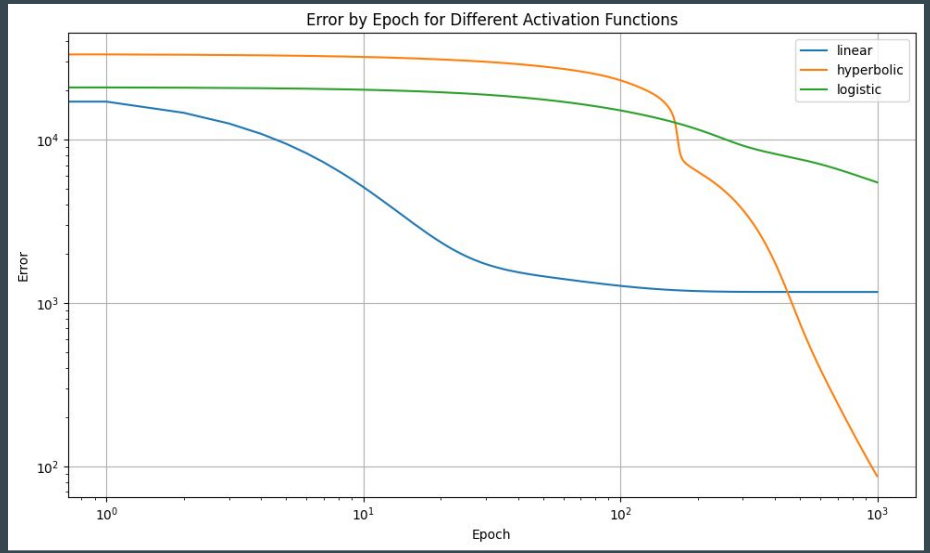
A tener en cuenta:

- Las funciones de activación que se utilizaran para los perceptrones no lineales son la tangente hiperbólica y la logística.
- Estas funciones tienen imágenes (-1, 1) y (0, 1) respectivamente, con los cual se debe hacer una normalización de los datos para obtener resultados adecuados.
- Para el cálculo de error se des-normalizan los datos con el objetivo de poder comparar adecuadamente los errores de los distintos perceptrones.

Perceptrones - Comparación

Para $\eta = 0.001$, $\beta = 1$ (para no lineales), 1000 épocas y pesos iniciales aleatorios.





Perceptrones - Comparación

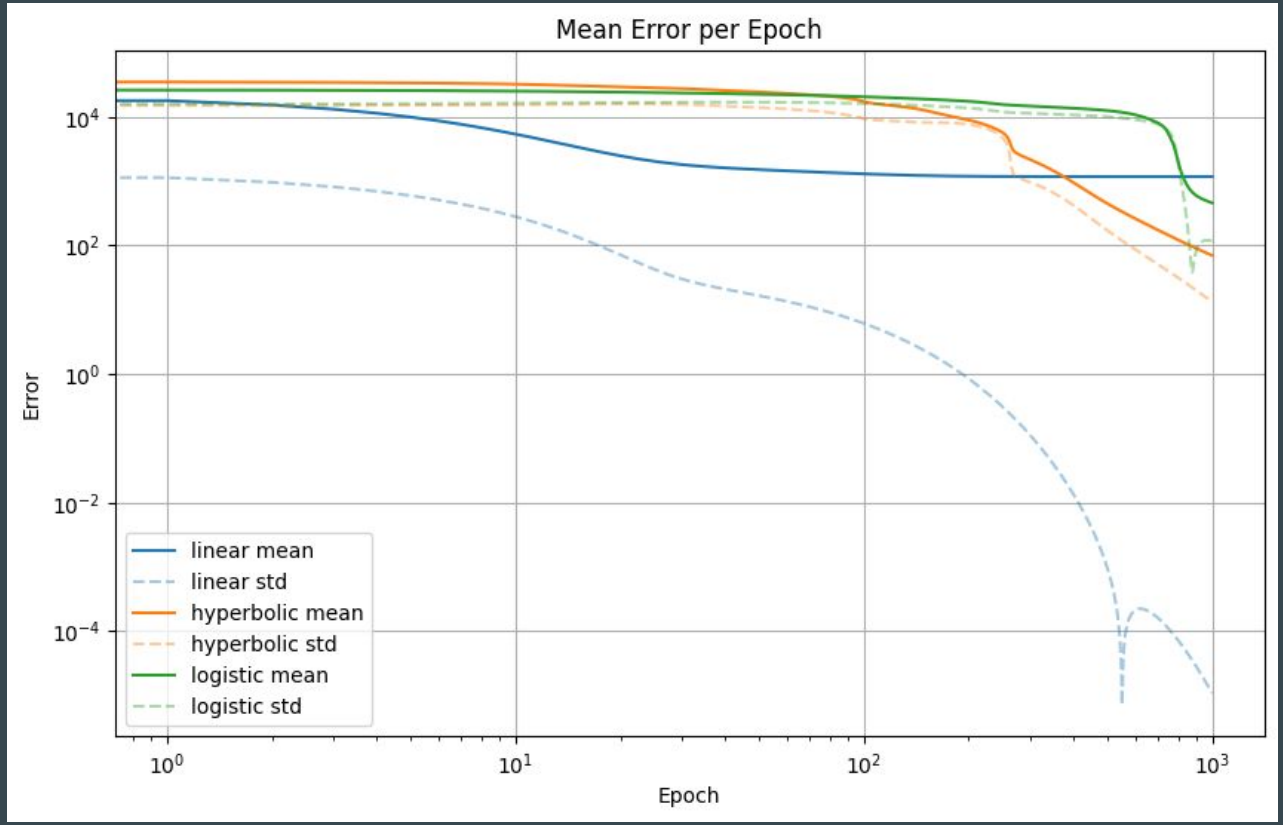
Promedio de 5 iteraciones

$E_{lin_min} = 1416$

$E_{hyp_min} = 92.57$

$E_{log_min} = 252.3$

Notar que el desvío estándar disminuye apreciablemente a medida que el error empieza a converger



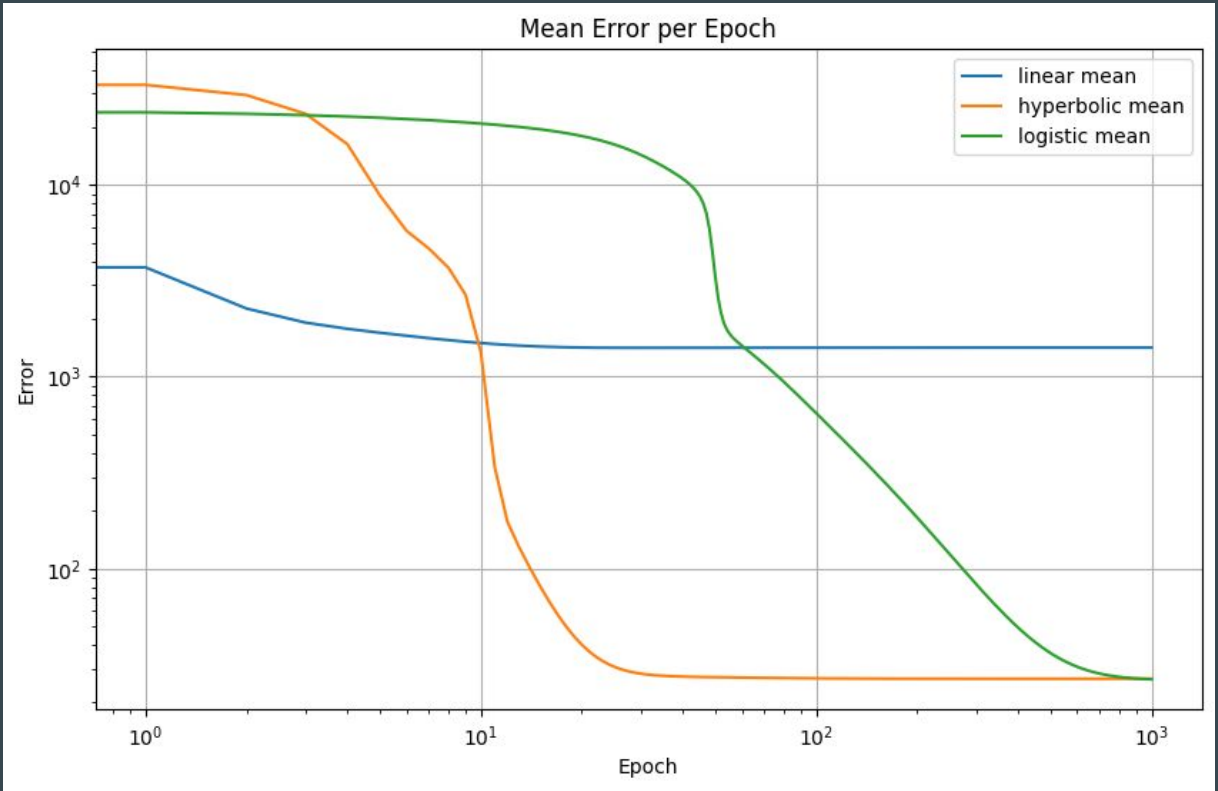
Perceptrones - Comparación

Para $\eta = 0.01$, $\beta = 1$ (para no lineales), 1000 épocas y pesos iniciales aleatorios.

$E_{lin_min} = 1416$

$E_{hyp_min} = 26.59$

$E_{log_min} = 26.41$

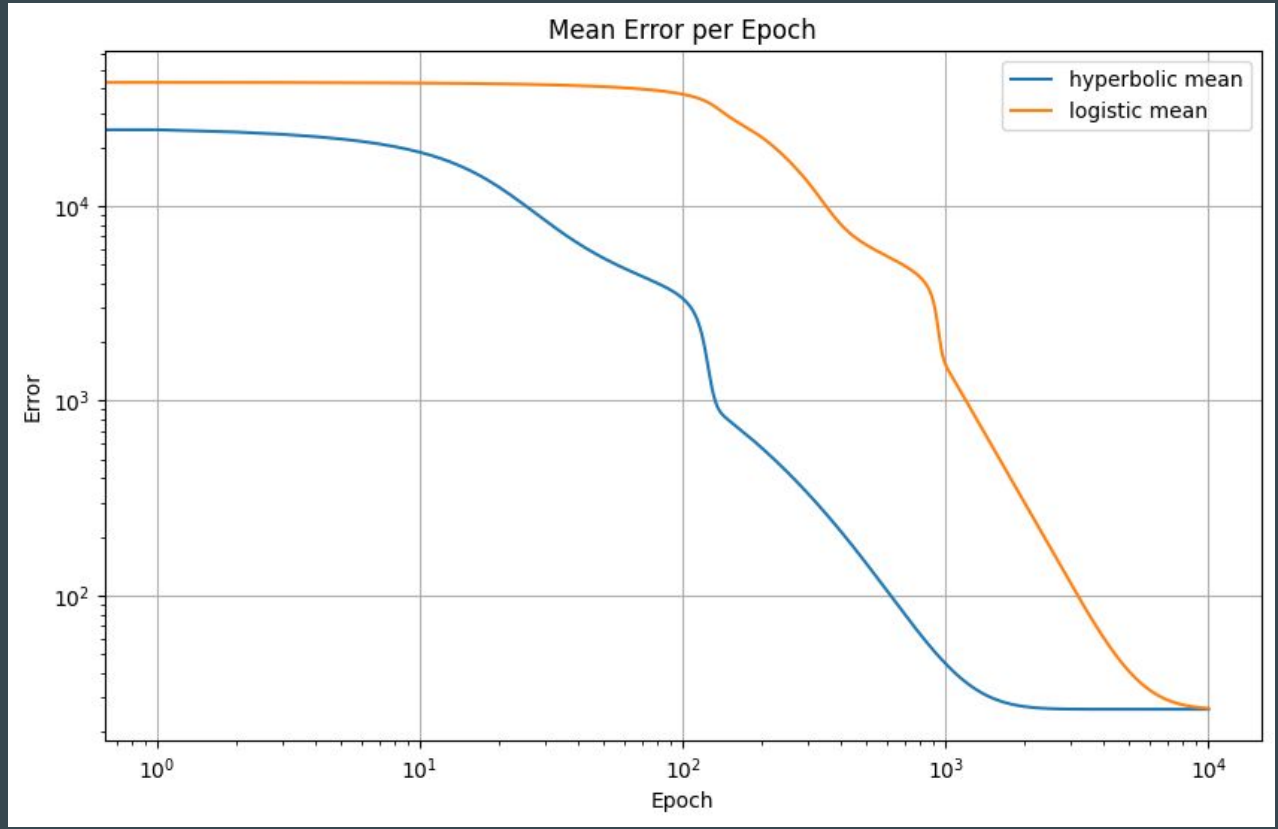


Perceptrones - Comparación

Para $\eta = 0.001$, $\beta = 1$ (para no lineales), 10000 épocas y pesos iniciales aleatorios.

$E_{hyp_min} = 26.12$

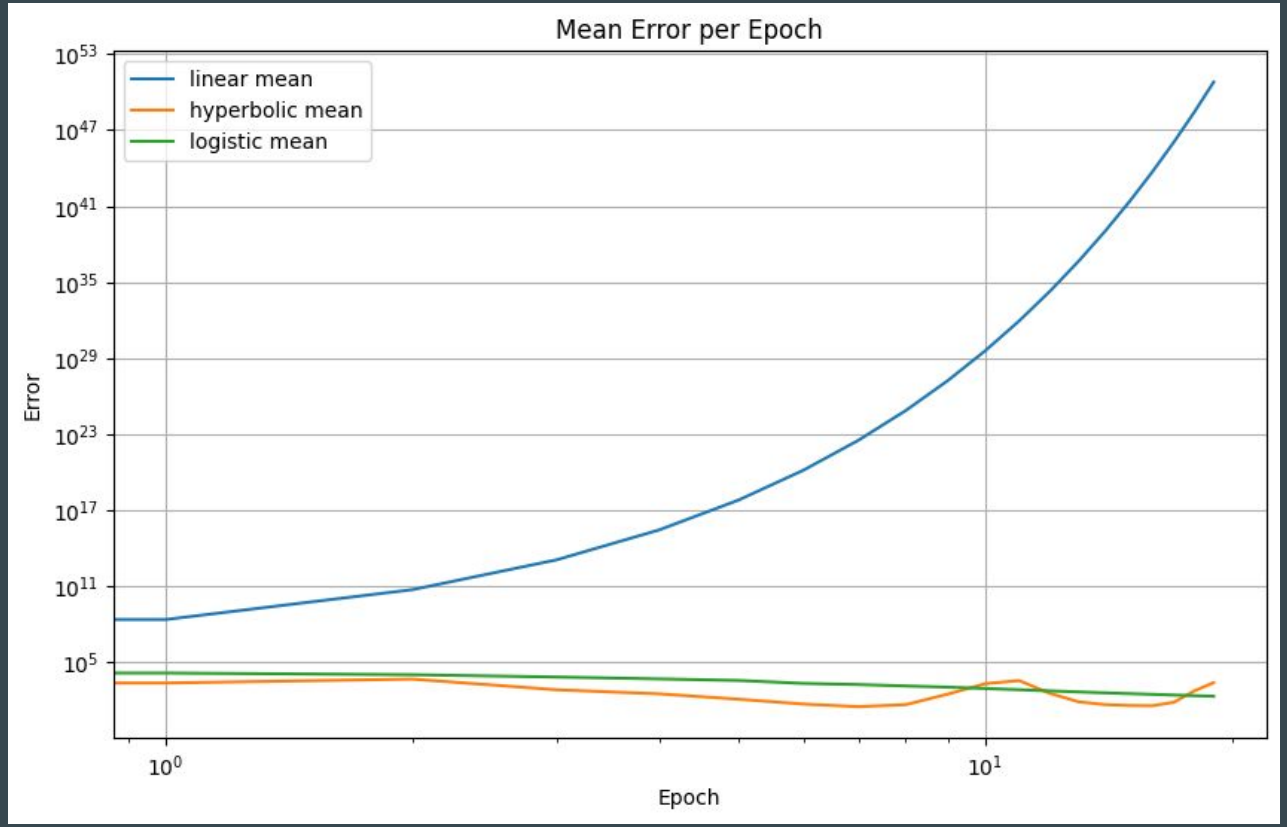
$E_{log_min} = 26.41$



Perceptrones - Comparación

Para $\eta = 0.1$, $\beta = 1$ (para no lineales), 20 épocas y pesos iniciales aleatorios.

Los errores se mantienen semi constantes o divergen.



Función de activación elegida

De los gráficos anteriores se puede observar que el error del perceptrón lineal converge órdenes de magnitud más rápidamente que para los no lineales, pero su error es mucho más grande.

Se puede concluir que esto se debe a que el dataset no tiene características de un sistema lineal.

Por ende la elección se reduce a alguno de los perceptrones no lineales. Los errores de ambos convergen a aproximadamente el mismo valor, pero el de la tangente hiperbólica lo hace un orden de magnitud más rápido.

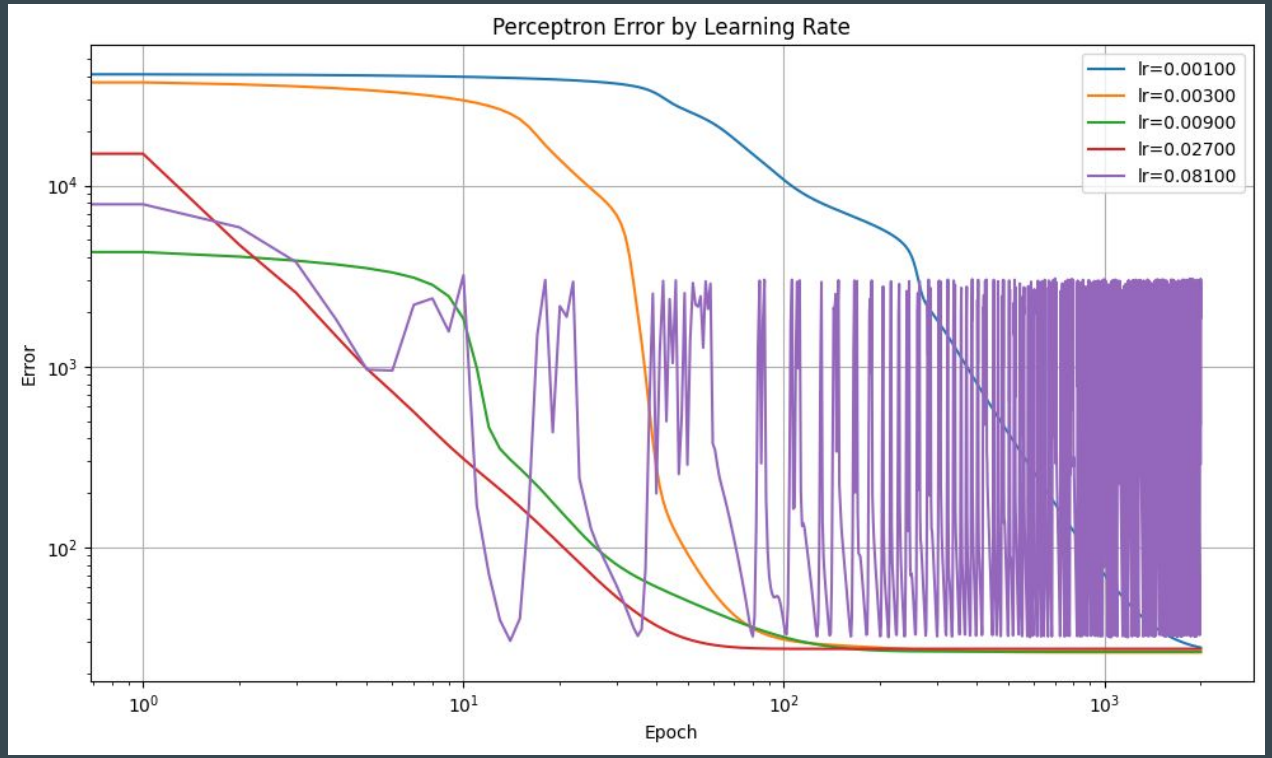
En conclusión la mejor función de activación para nuestro dataset es la **tangente hiperbólica** .

Perceptrón - Tangente Hiperbólica - Variación de tasa de aprendizaje

Para $\beta = 1$, 1000 épocas y pesos iniciales aleatorios.

Queremos encontrar una tasa de aprendizaje que nos dé buenos resultados lo más rápido posible. Tal como se vio antes, para valores relativamente grandes de la tasa la función no converge.

De ahora en adelante se analiza para valores menores a 0.05



Perceptrón - Tangente Hiperbólica

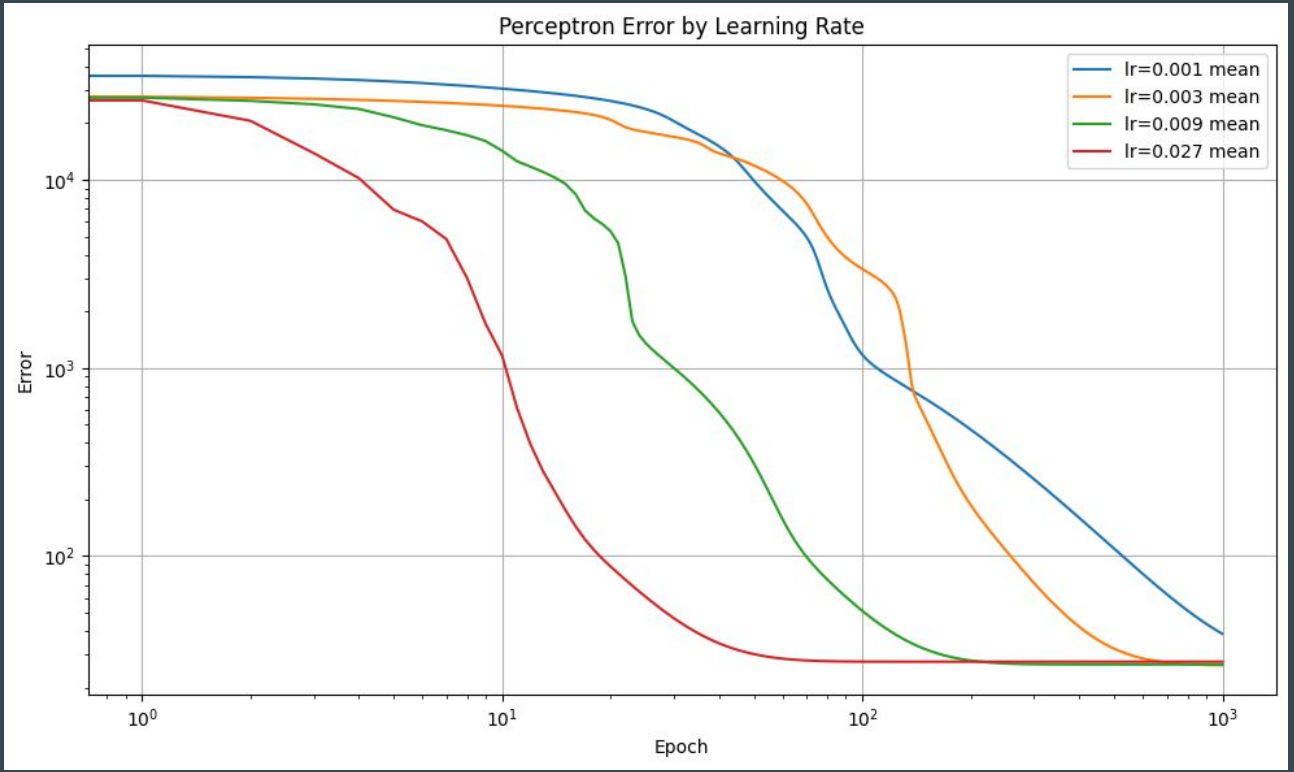
Para β = 1, 1000 épocas y pesos iniciales aleatorios. Promedio de 3 iteraciones

η: 0.001, min error: 38.63

η: 0.003, min error: 26.29

η: 0.009, min error: 26.53

η: 0.027, min error: 27.45



Perceptrón - Tangente Hiperbólica

La convergencia del error con $\eta = 0.001$ se da en alrededor de 2000 épocas y se consigue un error de aproximadamente 26.12, mientras que con $\eta = 0.009$ en 400 iteraciones se consigue un error de 26.53.

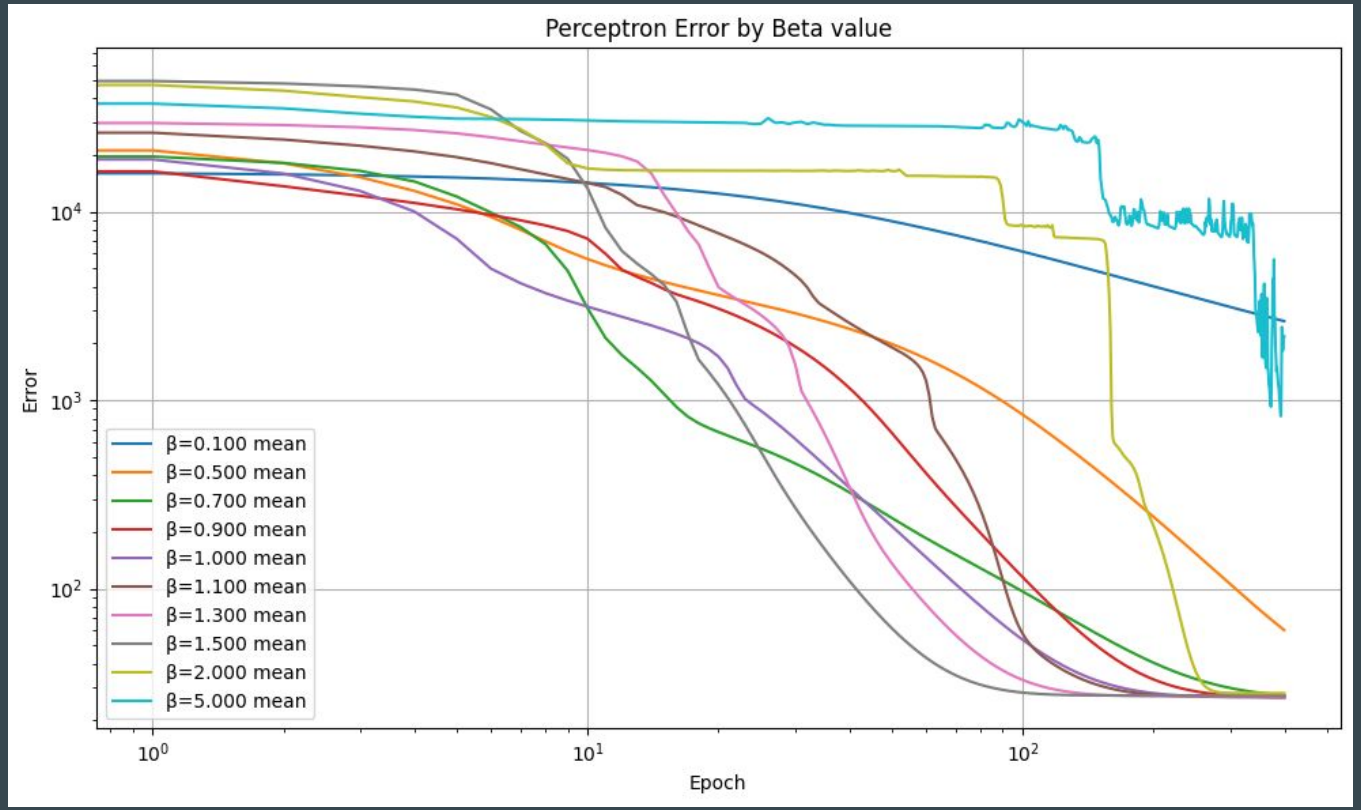
Osea un error 1.5% mayor pero en un 20% de las épocas, lo cual permitirá reducir mucho los tiempos de entrenamiento.

A partir de ahora usaremos $\eta = 0.009$.

Perceptrón - Tangente Hiperbólica - Variación de β

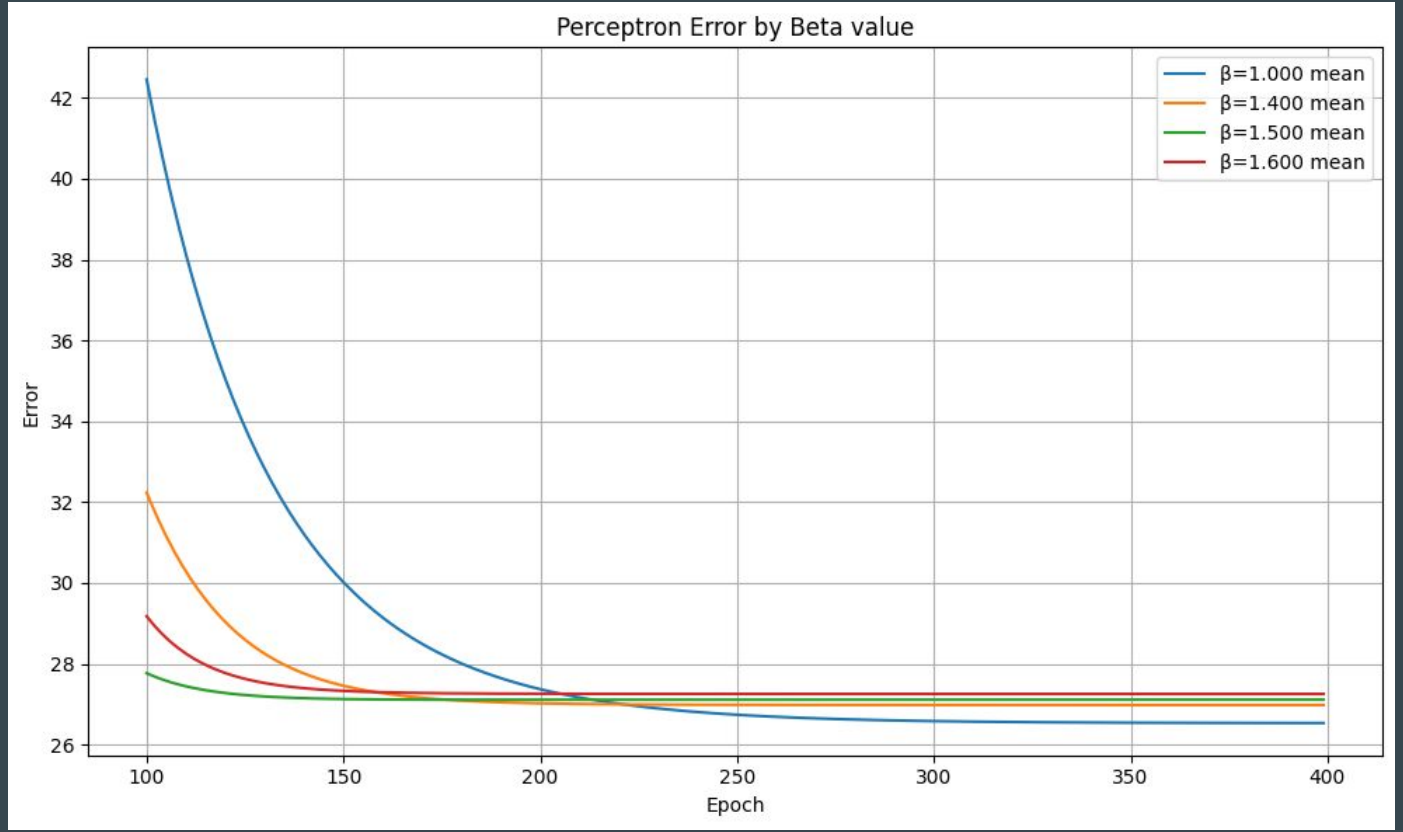
Para η = 0.009, 400 épocas y pesos iniciales aleatorios. Promedio de 3 iteraciones

Podemos ver que cuanto más grande β, para β < 2, más rápido converge el error. Mientras que para β >= 2 el modelo empieza a fallar.

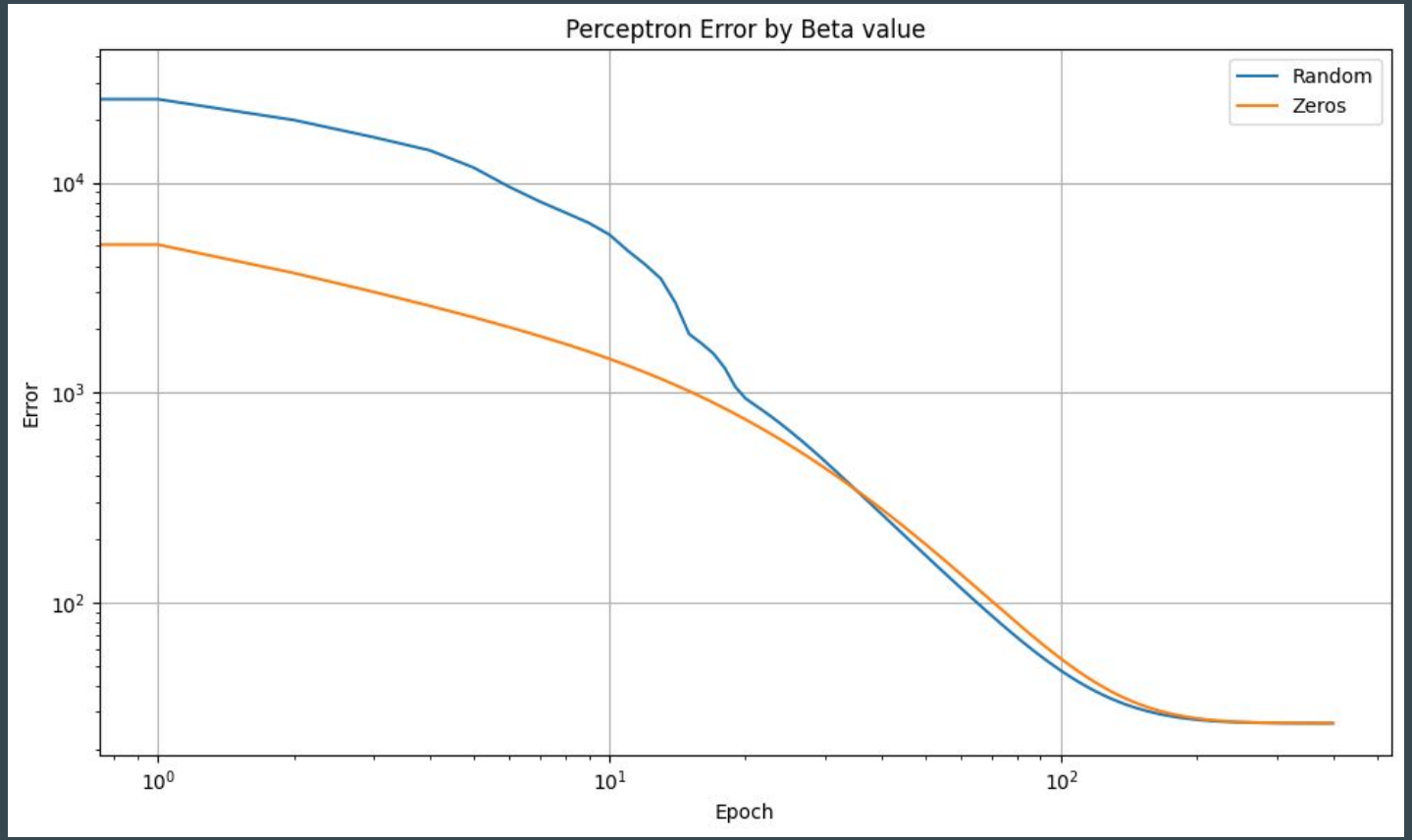


Perceptrón - Tangente Hiperbólica - Variación de β

Sin embargo como vemos acá, el error más pequeño es alcanzado con β=1



Perceptrón - Tangente Hiperbólica - Inicialización de pesos



Perceptrón - Tangente Hiperbólica - Estrategia cambio de pesos

$E_{min-1} = 26.54$

$E_{min-2} = 26.61$

$E_{min-3} = 26.40$

$E_{min-4} = 26.54$

$E_{min-5} = 26.33$

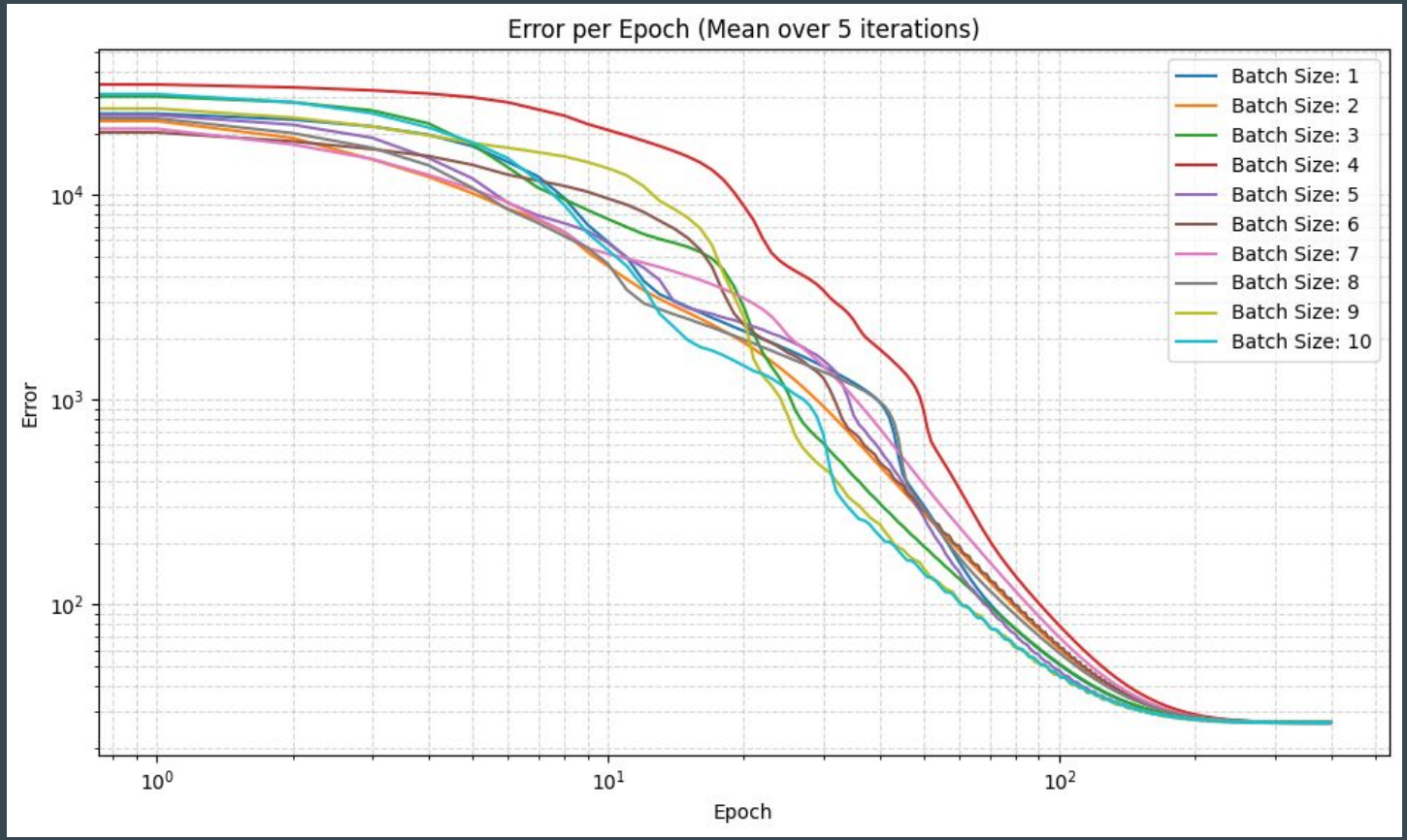
$E_{min-6} = 26.38$

$E_{min-7} = 26.41$

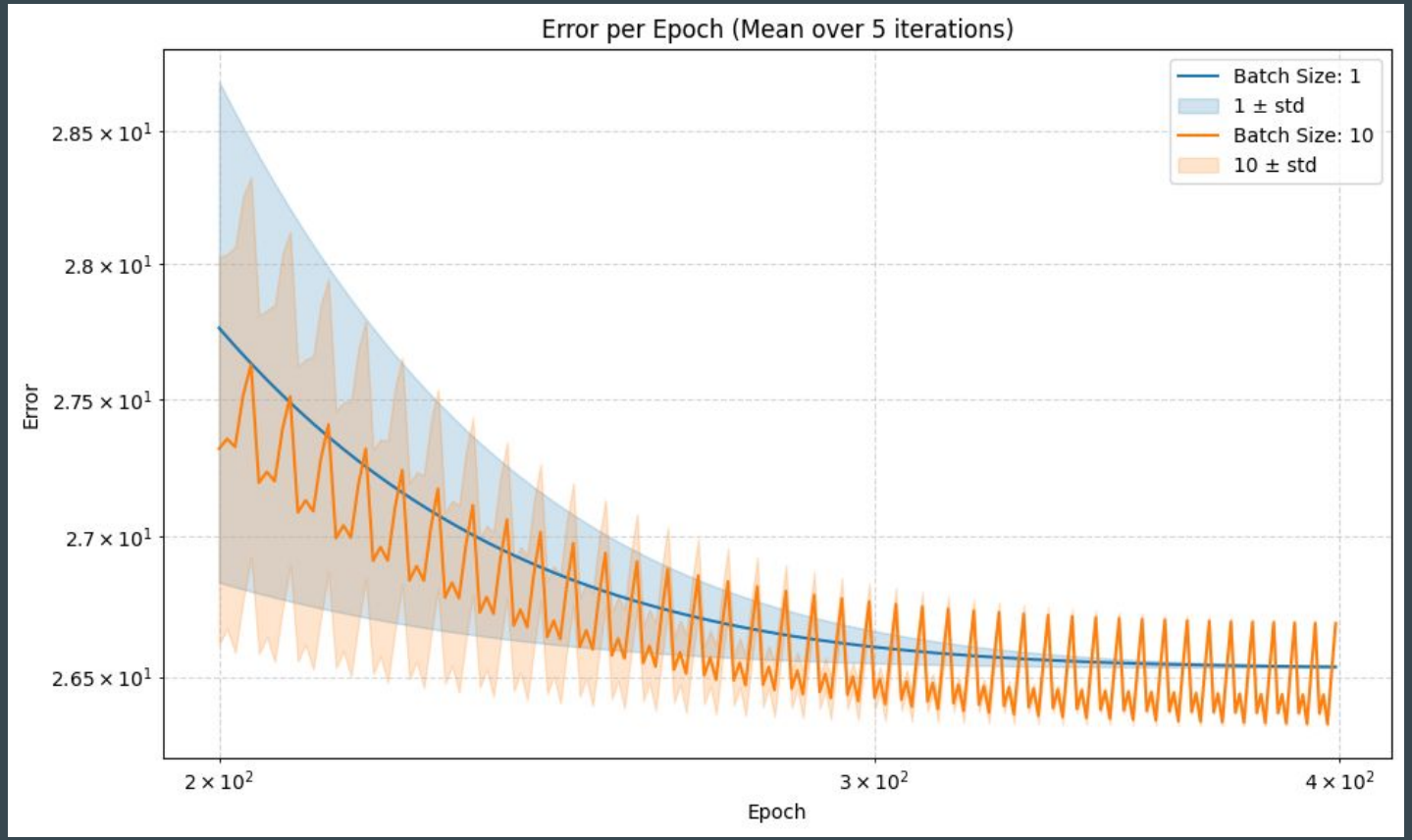
$E_{min-8} = 26.54$

$E_{min-9} = 26.32$

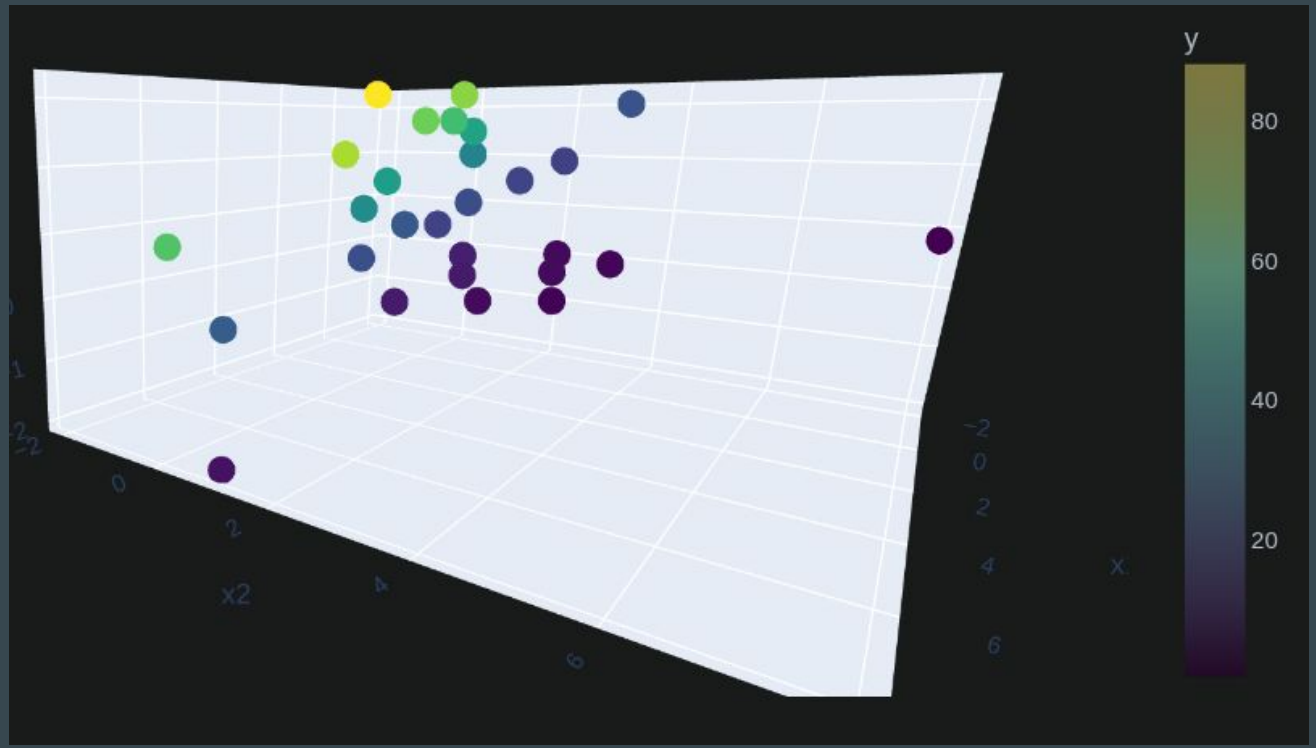
$E_{min-10} = 26.33$

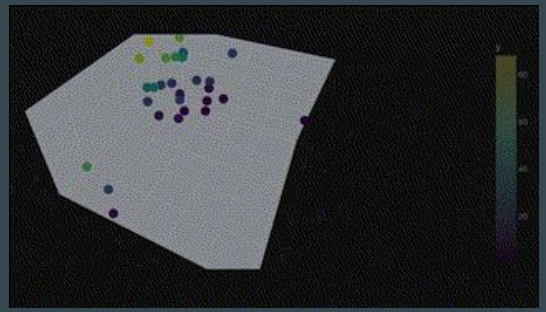


Perceptrón - Tangente Hiperbólica - Estrategia cambio de pesos



Perceptrón - Generalización - Estrategia



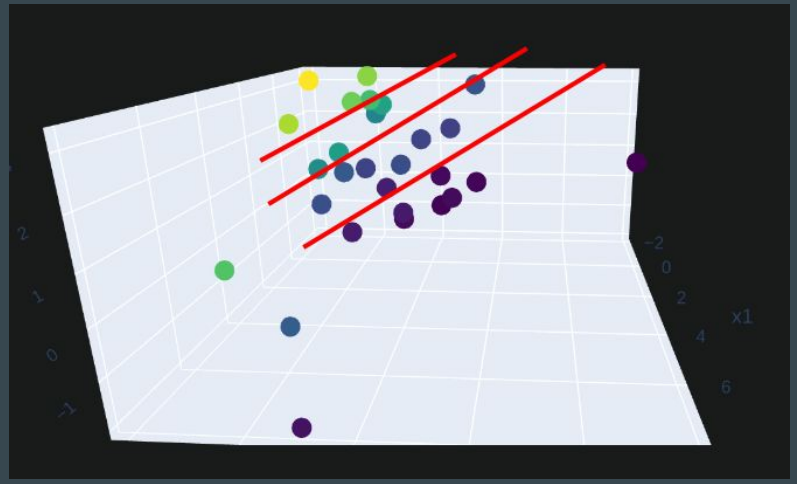


Perceptrón - Generalización - Estrategia

Se puede ver que a mayor x_3 y menor x_2 aumenta el valor de la salida, mientras que el valor en x_1 no parece tener mucha influencia.

Además se puede notar un agrupamiento grande de puntos y otros 4 puntos que parecerían outliers.

De acuerdo a esto, la estrategia utilizada será separar los datos en 4 categorías según el valor en Y y tomar datos al azar dentro de cada categoría. También se seleccionarán todos puntos outliers ya que estos nunca serán bien predecidos si no pertenecen al conjunto de entrenamiento.



Perceptrón - Generalización - Estrategia

x1	x2	x3	ev
0.000	-1.300	3.230	88.184
0.000	-2.000	2.000	76.852
-1.300	0.000	3.230	72.512
0.400	0.000	2.700	68.568
0.000	0.400	2.700	61.301

x1	x2	x3	ev
-0.500	0.600	2.500	51.000
1.800	0.000	1.600	49.000
1.200	-0.800	1.000	43.045
-2.000	0.000	2.000	40.131

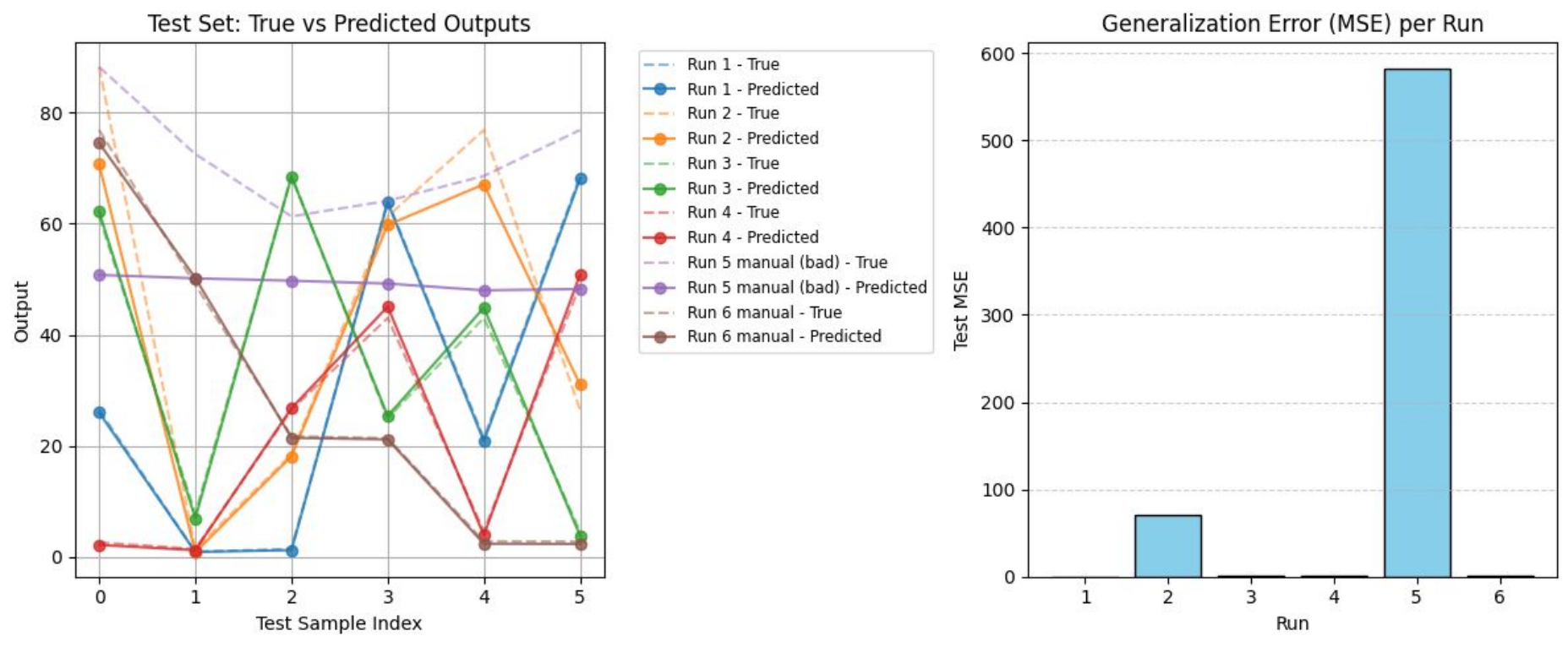
x1	x2	x3	ev
0.000	-0.500	0.600	24.974
-1.300	3.230	3.000	23.183
1.200	-0.800	0.000	21.755
1.800	1.600	1.300	21.417
0.000	1.800	1.600	18.543
-0.500	0.000	0.600	18.243
0.400	2.700	2.000	17.654

x1	x2	x3	ev
7.900	1.000	0.000	26.503
7.900	1.000	-2.000	4.653
7.900	0.000	1.000	64.107
0.000	7.900	1.000	0.320

x1	x2	x3	ev
-0.500	0.600	0.000	7.871
1.200	0.000	-0.800	7.176
1.800	1.600	0.000	6.914
0.000	1.200	-0.800	2.875
0.400	2.700	0.000	2.820
-2.000	2.000	0.000	2.660
-1.300	3.230	0.000	1.480
-2.000	2.000	-1.000	0.995

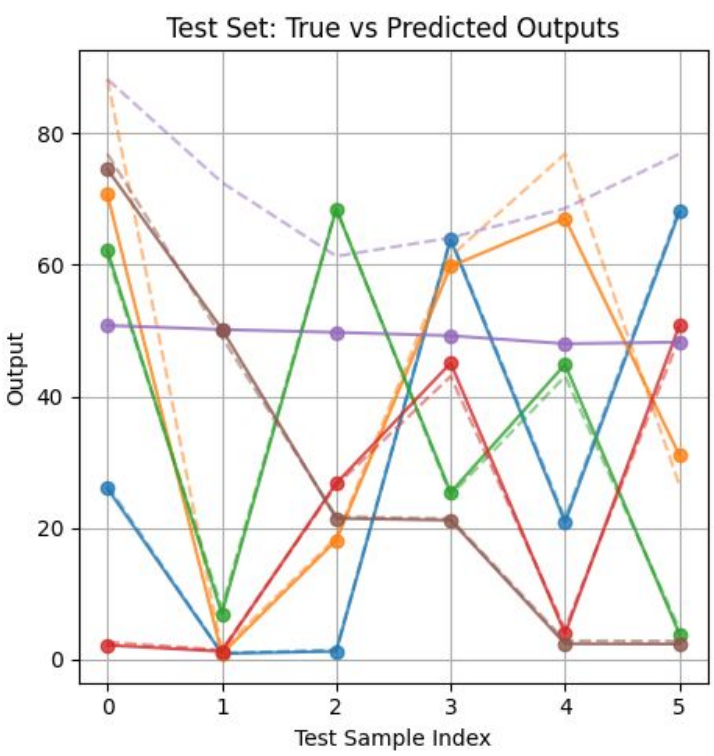
Perceptrón - Generalización - Estrategia

Para set de entrenamiento de 80%

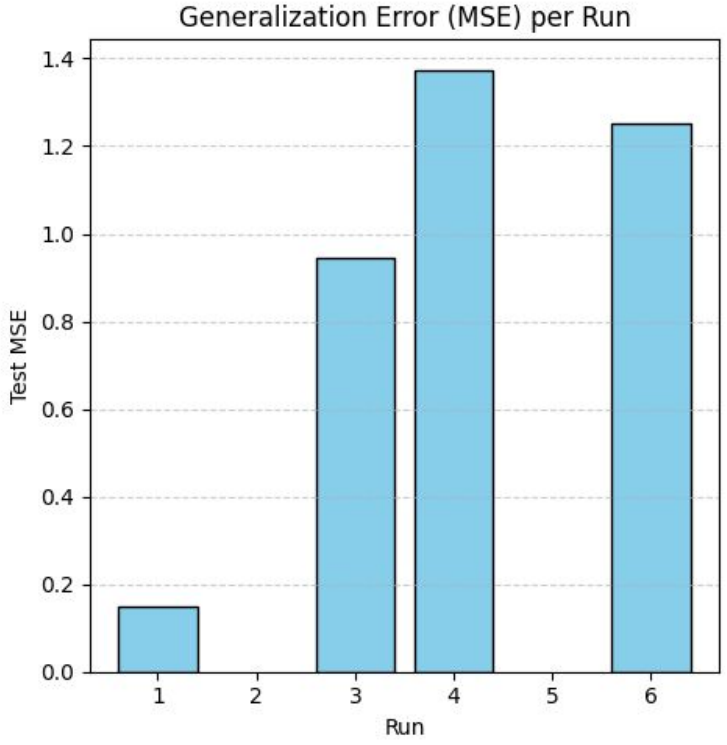


Perceptrón - Generalización - Estrategia

Para set de entrenamiento de 80%







Perceptrón - Generalización - Estrategia

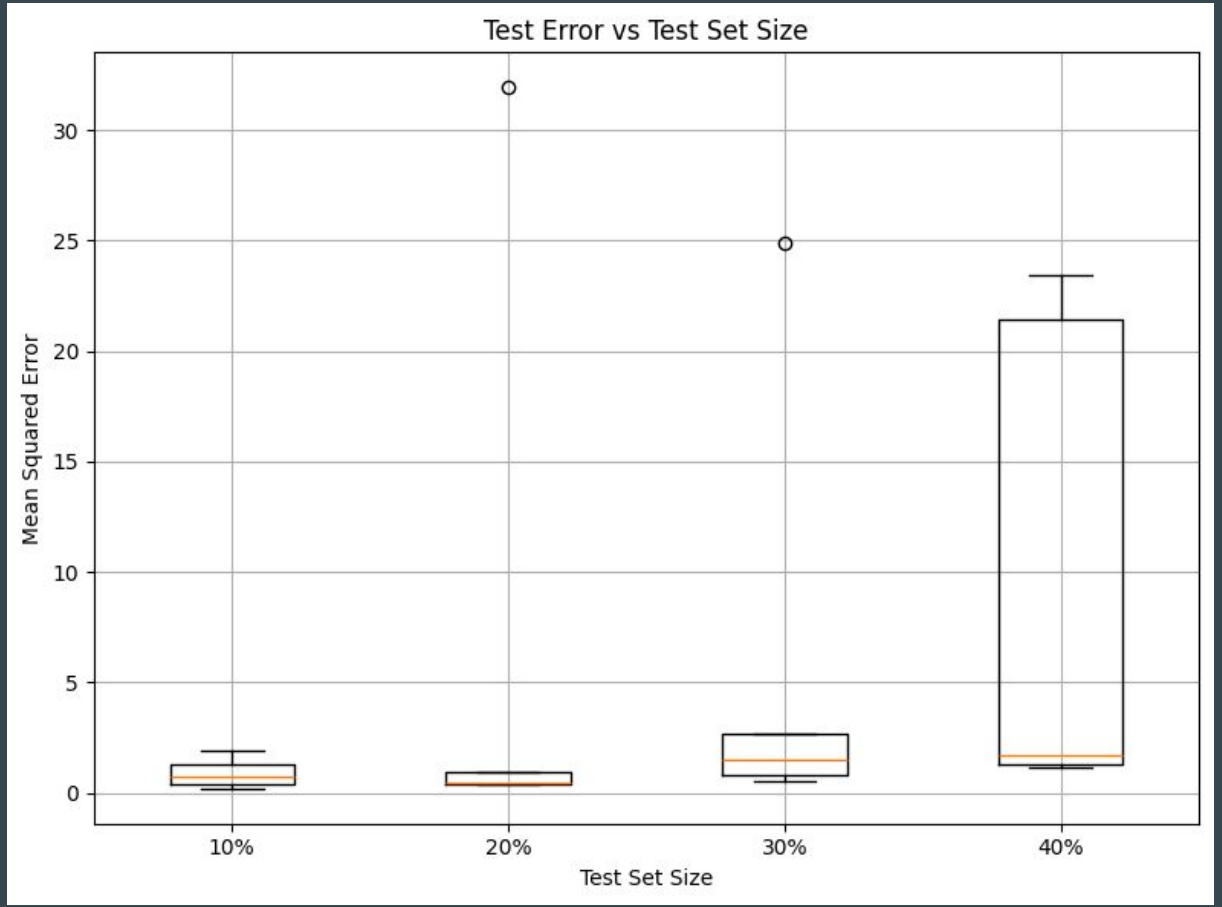
Se tomaron a sets de entrenamiento-testeo aleatorios, uno manual en base a la estrategia elegida, y otro manual donde para el set de testeo se tomaron solo datos con valor de salida alta (quedando pocos valores de salida alta en el set de entrenamiento).

Como se puede observar, usando sets aleatorios generalmente se consigue un error similar al set manual, aunque a veces puede ser bastante más alto (caso 2). Y se puede ver que el caso malo tiene efectivamente un error mucho más alto.

Perceptrón - Generalización - Porcentaje set entrenamiento

El modelo generaliza bien si se entrena en 70% o más de la data.

Tomaremos training sets de 80%.



Ejercicio 3 - Perceptrón Multicapa

Para el dado conjunto de datos buscamos comparar la soluciones según:

- La tasa de aprendizaje
- Forma de inicialización de pesos (aleatorios o en cero)
- El número de capas M
- El número de neuronas por capa
- Optimizador utilizado

Ejercicio 3 - Función XOR

- Retomando este problema, se decidió ver si ahora lo podíamos resolver utilizando el modelo de perceptrón multicapa
- Todo el dataset será usando para el training set
- Se buscó ver qué tan rápido converge con los distintos optimizadores
- Y si afecta la forma de inicializar los pesos

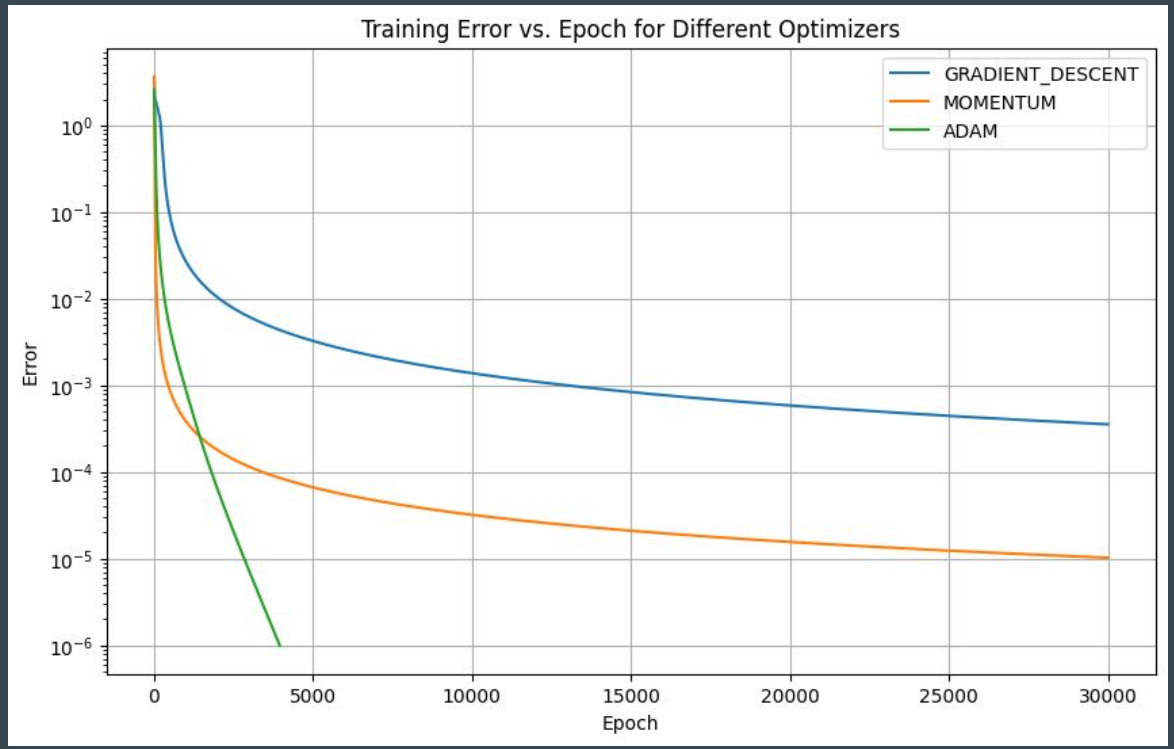
x1	x2	Expected Value
1	1	-1
1	-1	1
-1	1	1
-1	-1	-1



Ejercicio 3 - Función XOR

$\eta = 0.01$
min_error = 10^-6
max_epochs = 30000
function = tanh con $\beta = 1$
numpy_seed = 0
random_weight_ini = True

- En este ejemplo, se puede observar que el optimizador gradiente descendente cae en un mínimo local mucho más rápido
- Adam también destaca por ser el mejor en no caer en tantos mínimos locales



Ejercicio 3 - Función XOR

Outputs:

- Gradiente Descendente:

x1	x2	ev	predict
-1	1	1	0.988
1	-1	1	0.988
-1	-1	-1	-0.985
1	1	-1	-0.985

- Momentum:

-1	1	1	0.997
1	-1	1	0.998
-1	-1	-1	-0.998
1	1	-1	-0.998

- ADAM:

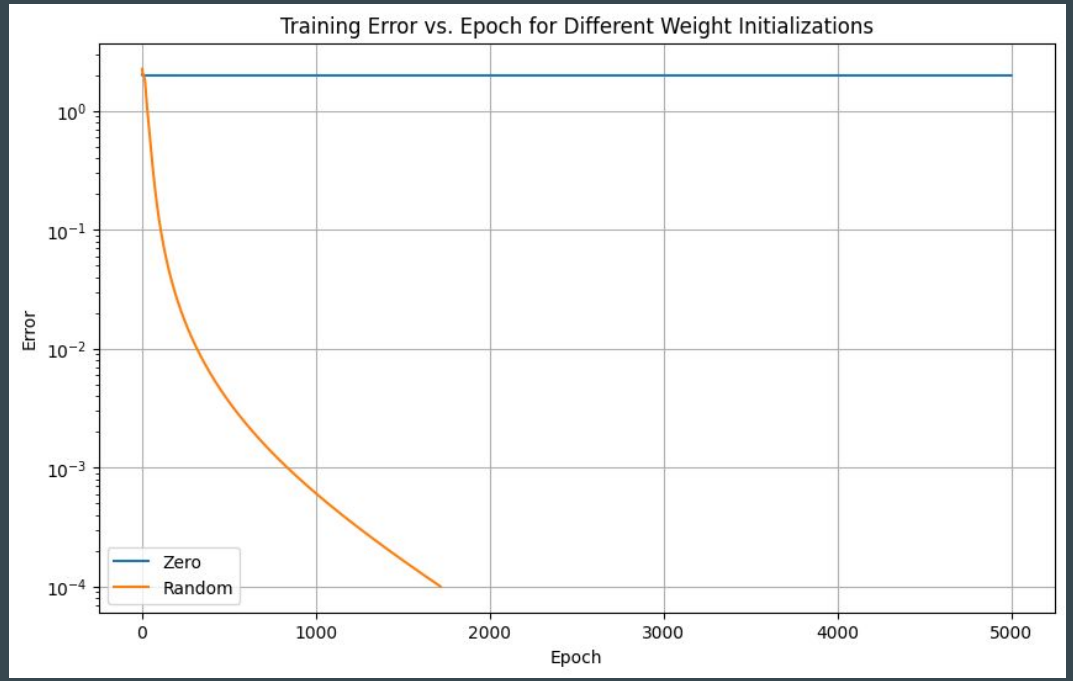
-1	1	1	0.999
1	-1	1	0.999
-1	-1	-1	-1
1	1	-1	-0.999

Ejercicio 3 - Función XOR - Inicialización de pesos

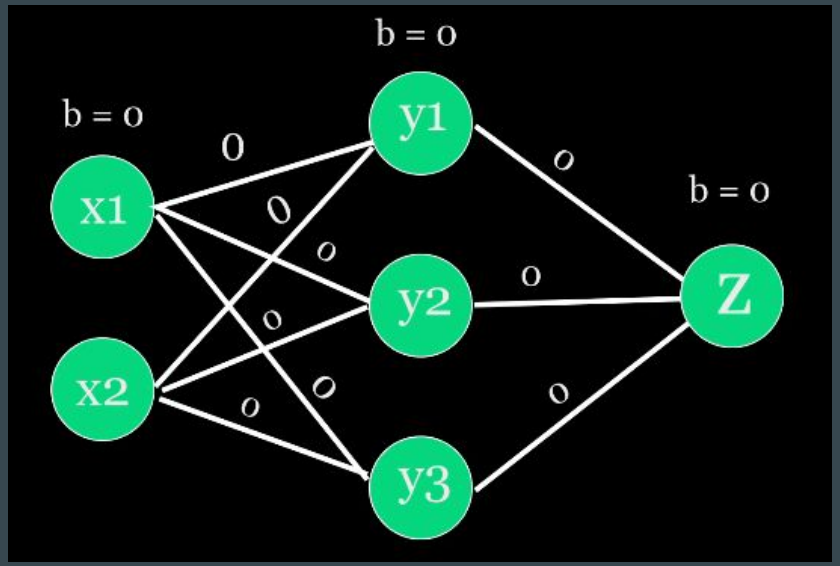


η = 0.01
min_error = 10^-4
max_epochs = 5000
function = tanh con β = 1
numpy_seed = 0
Optimizer = ADAM

¡¿Qué está pasando con los weights inicializados en cero?!



Ejercicio 3 - Función XOR - Inicialización de pesos



Feed-Forward Pass:

$$V_j^m = \theta\left(\sum_{k=1} V_k^{m-1} \cdot w_{jk}^m\right)$$

Gradiente:

$$\frac{\partial E}{\partial W_{ij}} = (\zeta_i - O_i)(-1)\,\theta'(h_i)(1)\,V_j^{M-1}$$

Fuente: Why Initialize a Neural Network with Random Weights

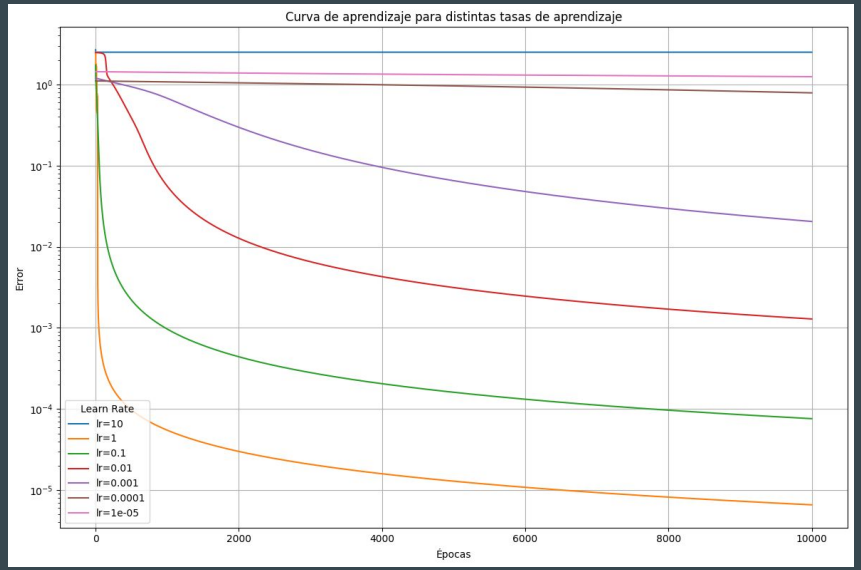
Ejercicio 3 - Paridad

- Para este problema se tiene de input 10 números de 0-9, cada uno con una frecuencia de una aparición, representados en bits con números 0 ó 1.
- Para este problema definimos de output “0” como par y “1” como impar
- Por la forma de output, decidimos utilizar la función logística, así no hay que realizar una normalización acorde
- Los inputs son los 35 bits del dibujo

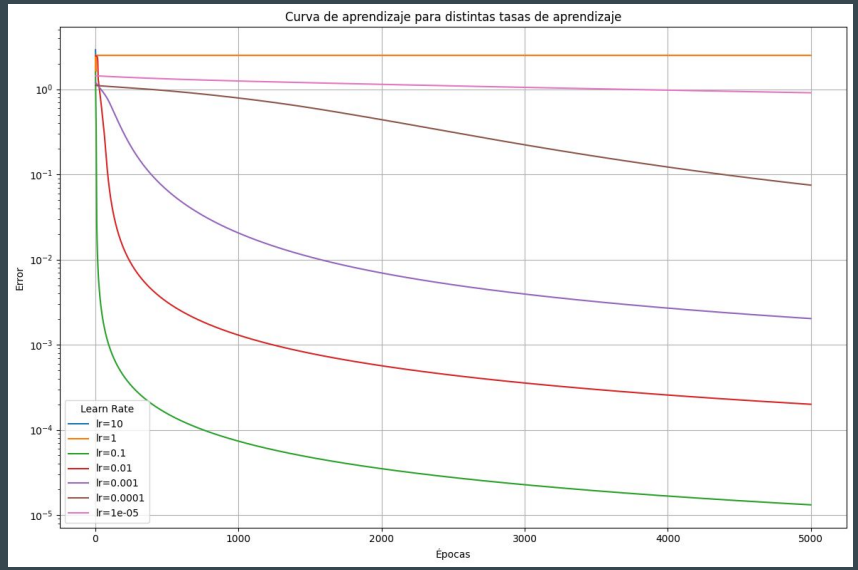


Ejercicio 3 - Paridad - Comparación de learn_rate

Gradiente Descendiente



Momentum



Se probó con learn_rate de 10, 1, 10^{-1}, ..., 10^{-5}

Arquitectura: [10, 10, 1]
y 35 inputs

min_error = 10^{-4}

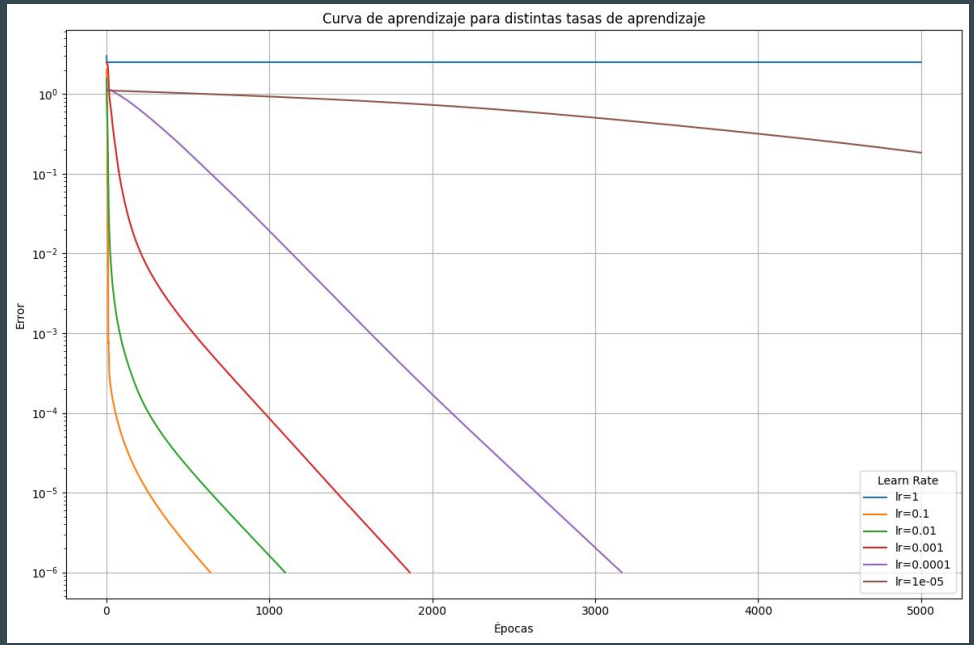
max_epochs = 5000

function = logistics con $\beta = 1$

numpy_seed = 0

Ejercicio 3 - Paridad - Comparación de learn_rate

ADAM



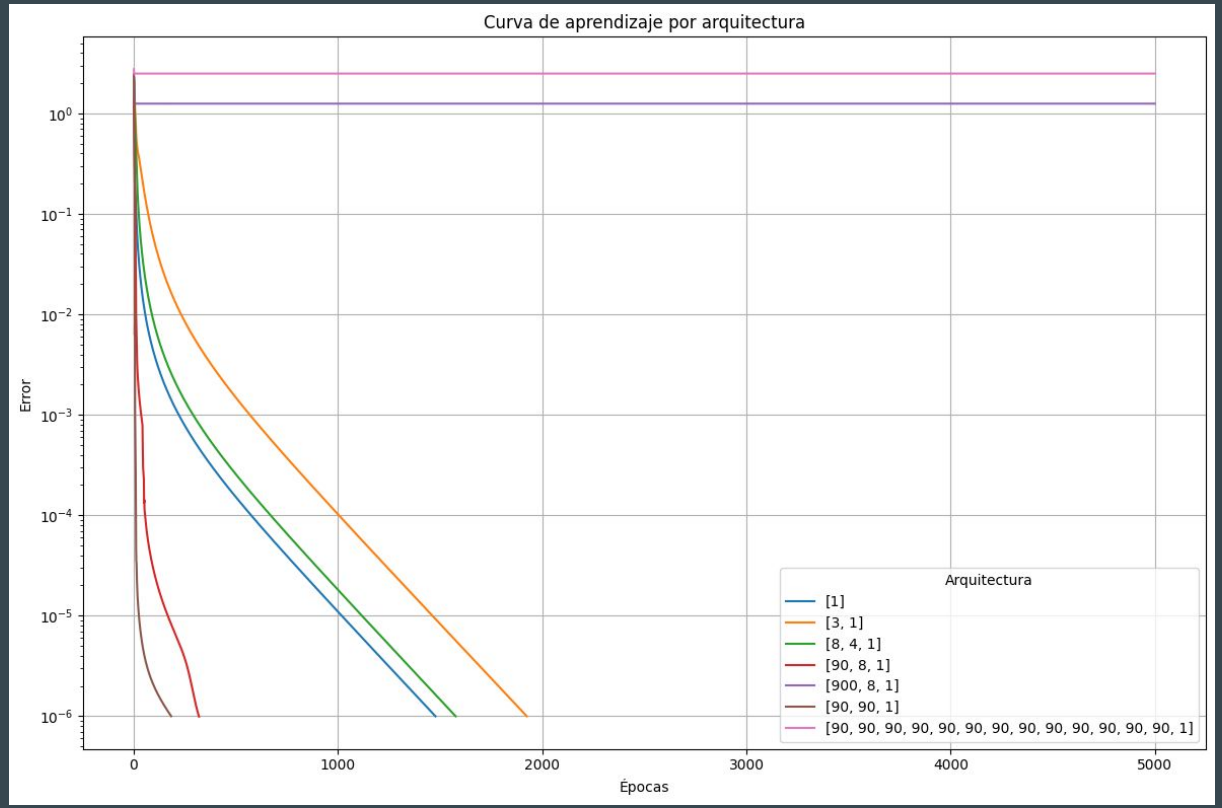
Se probó con learn_rate de 10, 1, 10^-1, ..., 10^-5

- Se observa que es más probable encontrar una buena solución rápido con ADAM

Ejercicio 3 - Paridad

$\eta = 0.01$
min_error = 10^-6
max_epochs = 5000
function = logistics con $\beta = 1$
numpy_seed = 0
Optimizer = ADAM

- Como se puede observar, más neuronas en una capa o más capas no necesariamente significa mejor convergencia. Incluso puede causar divergencia
- La arquitectura a elegir puede resultar muy arbitraria, depende de cosas como el tiempo de ejecución, velocidad de convergencia y cantidad de inputs/outputs



Los arreglos representan las arquitecturas de capas ocultas + la capa de salida

Ejercicio 3 - Dígitos

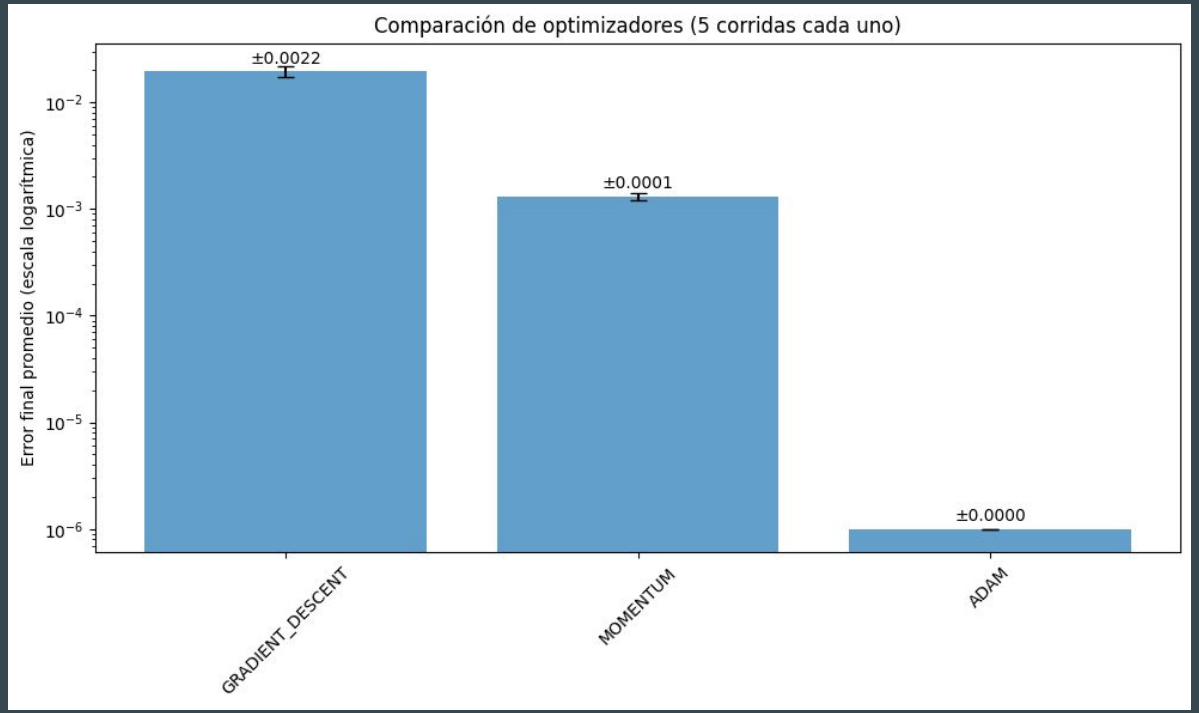
- Para este problema se tiene de input 10 números de 0-9, cada uno con una frecuencia de una aparición, representados en ascii con números 0 ó 1.
- Para este problema definimos como output un arreglo de size 10, con ceros en todas las posición menos en la posición del índice equivalente al carácter ascii representado, que tiene 1. Ejemplo 1: [1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- Por la forma de output, decidimos utilizar la función logística, ya que los valores dentro del arreglo estarán entre 0 - 1, definiendo qué tan probable es que sea un número u otro
- Los inputs son los 35 bits del dibujo

Ejercicio 3 - Dígitos - Comparación de optimizadores

35 inputs y Arquitectura: [16, 16, 10]

η = 0.01
min_error = 10^-6
max_epochs = 10000
function = logistics con β = 1
numpy_seed = random for each iter

- Se puede concluir que, en un límite de 10000 épocas, Adam consigue el mejor error promedio, como ya se fue viendo en los últimos ejercicios



Ejercicio 3 - Dígitos

- Sacamos el 2 y 8 del training set (20%), y veo qué predice el modelo cuando nunca vio estos datos:

```
prediction 2:[0.0, 0.0, 0.0, 0.12, 0.0, 0.0, 0.08, 0.25, 0.0, 0.0]
```

```
prediction 8:[0.13, 0.0, 0.0, 0.94, 0.0, 0.0, 0.01, 0.0, 0.0, 0.0]
```

- Sacamos el 2, 3 y 5 del training set (30%), y veo que predice el modelo cuándo nunca vio estos datos:

```
prediction 2:[0.0, 0.0, 0.0, 0.0, 0.01, 0.0, 0.0, 0.0, 0.01, 0.12]
```

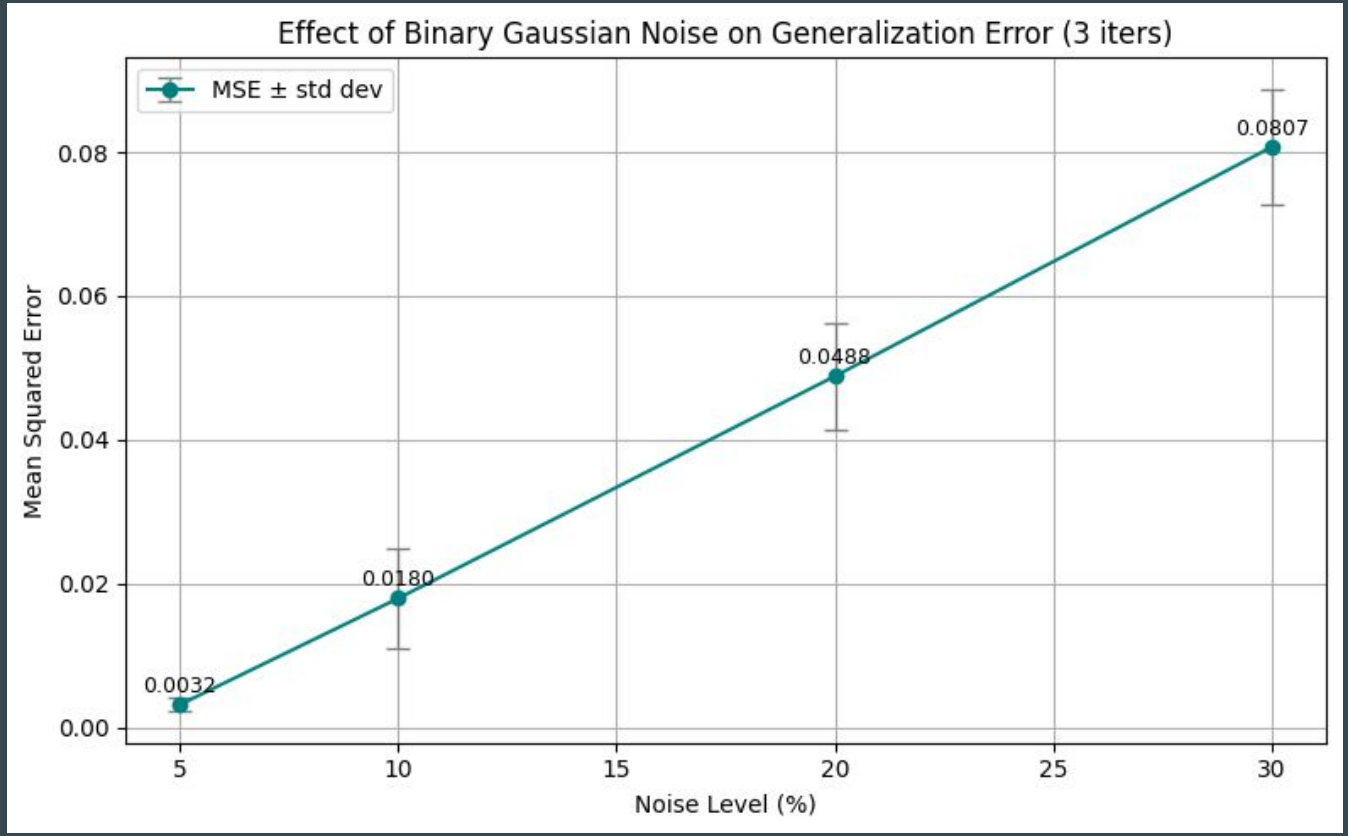
```
prediction 3:[0.01, 0.0, 0.0, 0.0, 0.01, 0.0, 0.0, 0.0, 0.96, 0.0]
```

```
prediction 5:[0.71, 0.49, 0.0, 0.0, 0.01, 0.0, 0.0, 0.0, 0.01, 0.0]
```

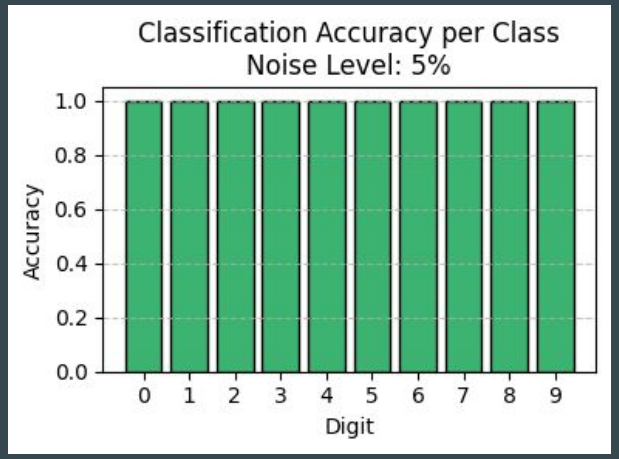
$\eta = 0.001$
E_{min} = 10^-4
$Epoch_{max}$ = 10000
Función logística con $\beta = 1$
numpy_seed = random for each iter
optimizer = ADAM

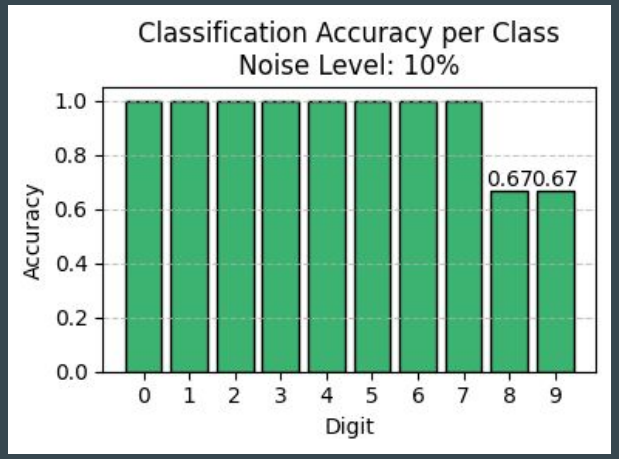
Ejercicio 3 - Ruido

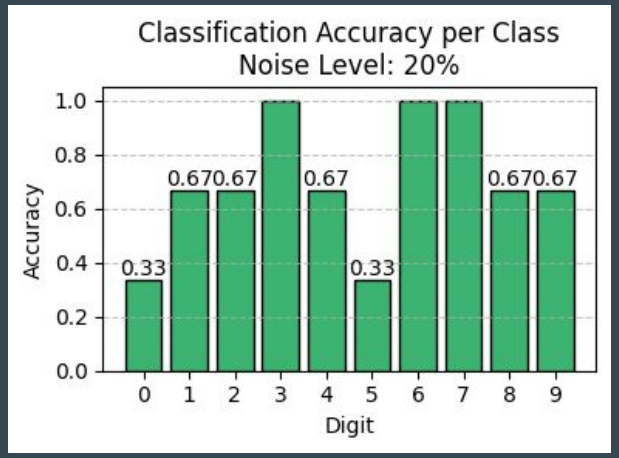
Entrenamos el perceptrón multicapa con el 100% de los datos originales y lo probamos con los datos más ruido gaussiano aplicado. Evaluamos distintos porcentajes de ruido.

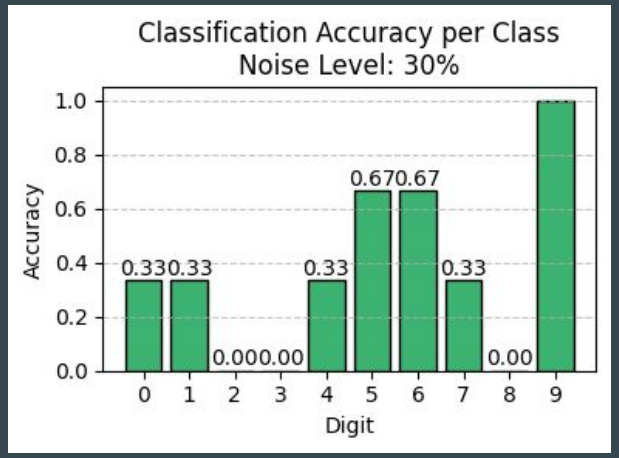


Ejercicio 3 - Ruido



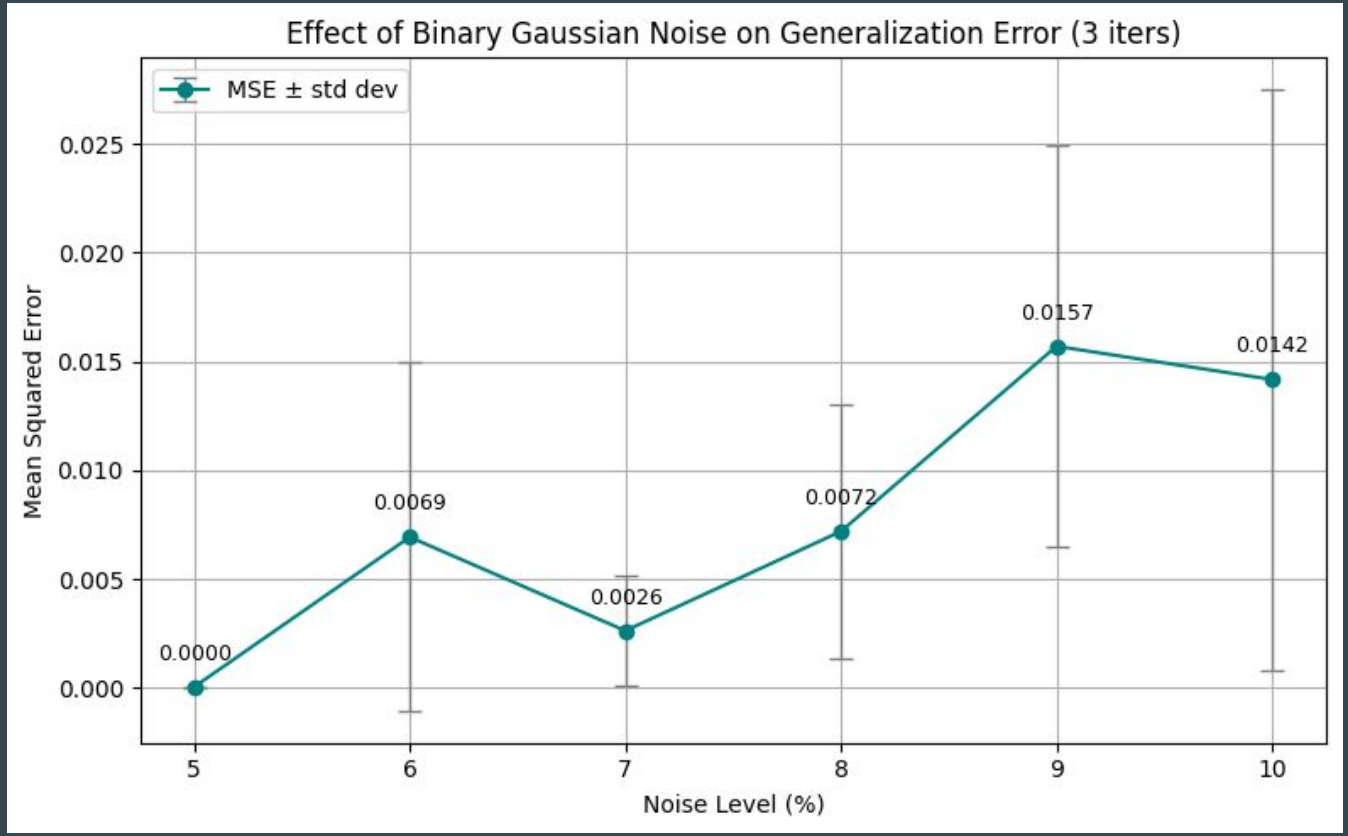




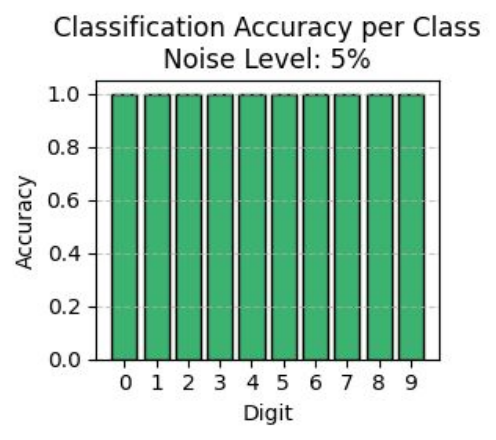


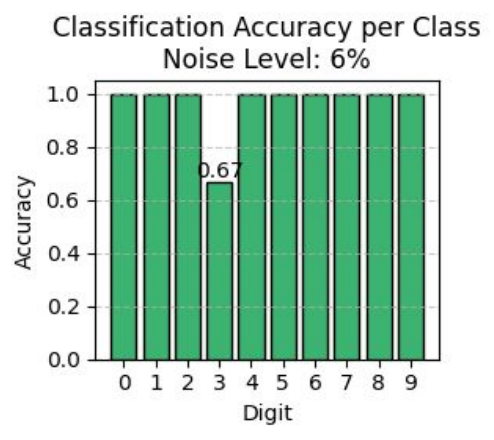
Ejercicio 3 - Ruido

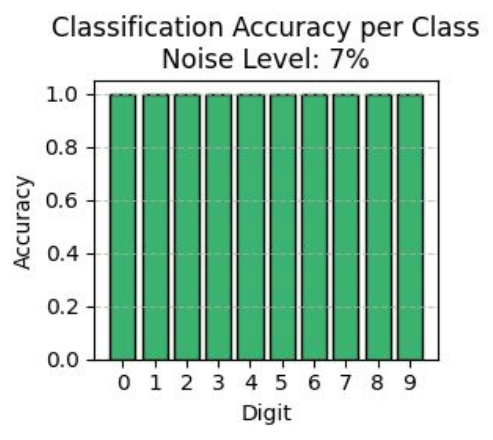
Analizamos para valores de ruido entre 5% y 10%.

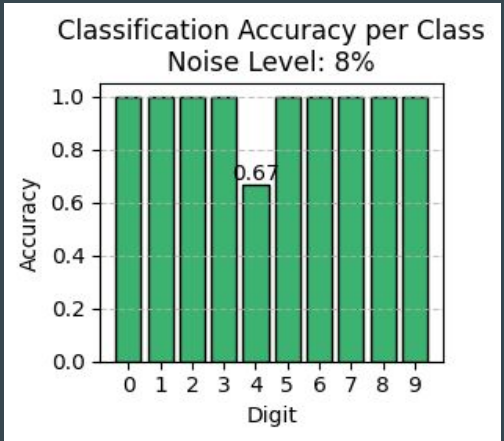


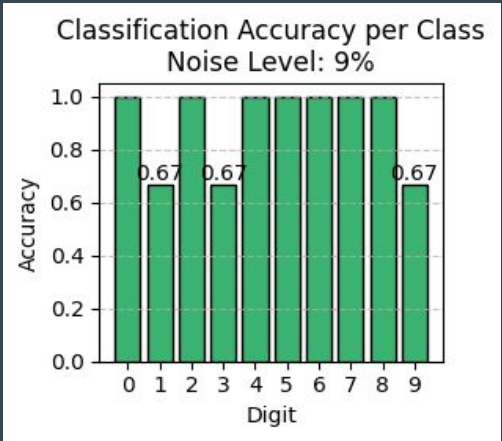
Ejercicio 3 - Ruido

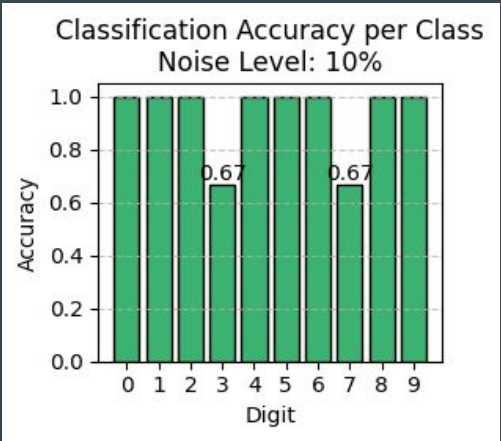












Ejercicio 3 - Ruido

- Para ruido menor o igual al 7% podemos tener una buena confianza de que el valor predicho será el correcto.
- Entre 8% y 10% es más probable empezar a tener algunos errores.
- Y si el ruido es mayor al 10% las predicciones ya son inútiles.

Esto último se puede deber a que nuestro dataset tiene una única representación para cada dígito, si el perceptrón fuera entrenado con variaciones de cada dígito quizá su capacidad de generalización ante ruido sería mejor.